Actividad etapa de Profundizacion : inicio en la creacion del proyecto-desarrollo de aplicación movil hibridas (tienda virtual)

Juan Francisco Alfonso Ruiz (arjuan@ucompensar.edu.co)

Ingeniería de Software

Fundación Universitaria U Compensar

Profesor:Julio cesar bolano peres

Desarrollo de aplicaciones moviles nativas

Octubre 20 del 2026

RESUMEN

El presente informe documenta el desarrollo de una aplicación móvil híbrida construida con Ionic, Angular y Capacitor, cuyo propósito es implementar una galería de fotos funcional con almacenamiento local persistente y un sistema de autenticación basado en tokens JWT. La aplicación permite a los usuarios registrarse con diferentes roles (comprador, vendedor y administrador), iniciar sesión con credenciales almacenadas localmente, y capturar fotografías que persisten entre sesiones. El proyecto fue desarrollado siguiendo los principios de la metodología ABP (Aprendizaje Basado en Proyectos) y fue migrado exitosamente a la plataforma Android para su validación en emulador. Los resultados demuestran el funcionamiento integral del flujo de autenticación, el almacenamiento local mediante Preferences y Filesystem, y la captura de imágenes con la cámara del dispositivo.

Palabras clave: Ionic, Angular, Capacitor, JWT, almacenamiento local, aplicación híbrida, galería de fotos.

1. INTRODUCCIÓN

El desarrollo de aplicaciones móviles ha evolucionado hacia soluciones multiplataforma que permiten a los desarrolladores construir una sola base de código y desplegarla en múltiples sistemas operativos. En este contexto, Ionic junto con Angular y Capacitor se han consolidado como herramientas fundamentales para el desarrollo de aplicaciones híbridas con apariencia y rendimiento similares a las nativas (Khanna, 2016).

La presente actividad tiene como propósito construir una aplicación con Ionic que implemente una galería de fotos utilizando la cámara del dispositivo, aplicando los conceptos de almacenamiento local y arquitectura JWT (JSON Web Token) para la autenticación de usuarios. El

proyecto constituye la base tecnológica para el desarrollo posterior de una tienda virtual de productos artesanales, cuyo diseño completo se encuentra documentado en la etapa de contextualización.

Los objetivos específicos abordados fueron: (1) configurar un proyecto Ionic con Angular y Capacitor; (2) instalar y configurar los plugins requeridos para cámara, sistema de archivos y almacenamiento; (3) implementar un servicio de fotografías con persistencia local; (4) construir un flujo completo de autenticación con JWT y múltiples roles; (5) migrar la aplicación a Android y validarla en emulador; y (6) documentar el proceso mediante pantallazos.

2. MARCO TEÓRICO

2.1 Ionic Framework

Ionic es un framework de código abierto orientado a la construcción de aplicaciones móviles híbridas. Proporciona una biblioteca de componentes de interfaz de usuario con apariencia nativa que se adapta automáticamente al sistema operativo en el que se ejecuta (iOS, Android o web). Los componentes de Ionic están construidos sobre tecnologías web estándar (HTML, CSS y JavaScript) y se integran con frameworks como Angular, React o Vue (Khanna, 2016).

2.2 Angular

Angular es un framework de desarrollo frontend mantenido por Google. Su arquitectura basada en componentes, su sistema de inyección de dependencias y su soporte para TypeScript lo convierten en una herramienta robusta para construir aplicaciones escalables. Desde la versión 17, Angular introdujo los "Standalone Components", que eliminan la necesidad de los módulos (NgModules) y simplifican la estructura del proyecto (Angular Framework, s.f.).

Adicionalmente, Angular 17 incorporó el control flow integrado (@if, @for, @switch), que reemplaza a las directivas estructurales clásicas (*ngIf, *ngFor) y mejora el rendimiento del renderizado.

2.3 Capacitor

Capacitor es el runtime nativo desarrollado por el equipo de Ionic que permite acceder a las APIs del dispositivo (cámara, sistema de archivos, geolocalización, notificaciones) desde una aplicación web. A diferencia de Cordova, Capacitor fue diseñado desde cero con un enfoque moderno y ofrece un mayor control sobre el proyecto nativo (Ionic Framework, s.f.).

2.4 Almacenamiento local

Las aplicaciones móviles híbridas disponen de múltiples mecanismos de almacenamiento local. Los utilizados en esta actividad son:

- Preferences: API de Capacitor para almacenar pares clave-valor de forma persistente. Ideal para datos pequeños como tokens, configuraciones o listas de metadatos.
- Filesystem: API de Capacitor para leer y escribir archivos en el sistema de archivos del dispositivo. Utilizada para imágenes, videos y documentos.

En la web, ambos plugins son emulados por Capacitor usando IndexedDB, mientras que en Android se traducen a SharedPreferences y al sistema de archivos privado de la aplicación, respectivamente.

2.5 JSON Web Token (JWT)

JWT es un estándar abierto (RFC 7519) que define un mecanismo compacto y autocontenido para transmitir información entre dos partes de forma segura. Un JWT está compuesto por tres partes separadas por puntos:

header.payload.signature

Header: algoritmo de firma y tipo de token.

Payload: "claims" o declaraciones (usuario, rol, fecha de expiración, etc.).

Signature: firma digital generada con una clave secreta que garantiza la integridad del token.

En el flujo de autenticación, el servidor entrega el JWT tras un login exitoso, y el cliente lo adjunta en cada petición HTTP mediante la cabecera Authorization: Bearer <token>. El backend verifica la firma y extrae la identidad del usuario sin necesidad de consultar la base de datos en cada petición (Recio García, 2018).

2.6 Metodología ABP

El Aprendizaje Basado en Proyectos (ABP) es una metodología activa que orienta el proceso formativo a partir del desarrollo de un producto real. Esta metodología permite integrar conocimientos teóricos con la resolución de problemas prácticos, fomentando el aprendizaje autónomo y el trabajo iterativo (Lasa Gómez et al., 2017).

3. DESARROLLO

3.1 Configuración del entorno

El primer paso consistió en la instalación de las herramientas necesarias en un sistema operativo Ubuntu. Se instalaron:

Node.js (v24.22.1): runtime de JavaScript que permite ejecutar herramientas como npm e Ionic CLI.

npm (v11.19.1): gestor de paquetes de Node.js.

Ionic CLI: herramienta de línea de comandos para crear y gestionar proyectos Ionic.

Android Studio: IDE oficial para el desarrollo Android, necesario para compilar y ejecutar la versión nativa.

```
aplicacion)$ node
Welcome to Node.js v24.22.1.
Type ".help" for more information.
> help
Uncaught ReferenceError: help is not defined
> .help
.break    Sometimes you get stuck, this gets you out
.clear    Alias for .break
.editor   Enter editor mode
.exit     Exit the REPL
.help     Print this help message
.load     Load JS from a file into the REPL session
.save     Save all evaluated commands in this REPL session to a file
```

```
NPM(1)                                                                  NPM(1)

NAME
       npm - javascript package manager

   Synopsis
          npm

       Note: This command is unaware of workspaces.

   Version
       11.19.1

   Description
       npm is the package manager for the Node JavaScript platform. It puts modules in place so that node can find them, and manages dependency conflicts intelligently.

       It is extremely configurable to support a variety of use cases. Most commonly, you use it to publish, discover, install, and develop node programs.

       Run npm help to get a list of available commands.

   Important
       npm comes preconfigured to use npm's public registry at https://registry.npmjs.org by default. Use of the npm public registry is subject to terms of use available at
       https://docs.npmjs.com/policies/terms.

       You can configure npm to use any compatible registry you like, and even run your own registry. Use of someone else's registry is governed by their terms of use.

   Introduction
       You probably got npm because you want to install stuff.

       The very first thing you will most likely want to run in any node program is npm install to install its dependencies.

       You can also run npm install blerg to install the latest version of "blerg". Check out npm help install for more info. It can do a lot of stuff.

       Use the npm search command to show everything that's available in the public registry. Use npm ls to show everything you've installed.

   Dependencies
       If a package lists a dependency using a git URL, npm will install that dependency using the \fBgit\fR (https://github.com/git-guides/install-git) command and will generate an er-
       ror if it is not installed.

       If one of the packages npm tries to install is a native node module and requires compiling of C++ Code, npm will use node-gyp (https://github.com/nodejs/node-gyp) for that task.
       For a Unix system, node-gyp (https://github.com/nodejs/node-gyp) needs Python, make and a buildchain like GCC. On Windows, Python and Microsoft Visual Studio C++ are needed. For
       more information visit the node-gyp repository (https://github.com/nodejs/node-gyp) and the node-gyp Wiki (https://github.com/nodejs/node-gyp/wiki).

   Directories
       See npm help folders to learn about where npm puts stuff.
Manual page npm.1 line 1 (press h for help or q to quit)
```

```
patch@pop-os:~/Desktop/desarrolo aplicaciones hivridas(ionic)/miTiendaArtesanal(aplicacion)$ ionic --vercion


  _                 _
 (_) ___  _ __ (_) ___
 | |/ _ \| '_ \| |/ __|
 | | (_) | | | | | (__
 |_|\___/|_| |_|_|\___| CLI 7.2.1


 Usage:

   $ ionic <command> [<args>] [--help] [--verbose] [--quiet] [--no-interactive] [--no-color] [--confirm] [options]

 Global Commands:

   completion ..................... (experimental) Enables tab-completion for Ionic CLI commands.
   config <subcommand> ............ Manage CLI and project config values (subcommands: get, set, unset)
   info ........................... Print project, system, and environment information
   init ........................... (beta) Initialize existing projects with Ionic
   live-update <subcommand> ....... (paid) Ionic Live Updates functionality (subcommands: manifest)
   login .......................... Log in to Ionic
   logout ......................... Log out of Ionic
   signup ......................... Create an Ionic account
   ssh <subcommand> ............... (deprecated) Commands for configuring SSH keys (subcommands: add, delete,
                                    generate, list, setup, use)
   start .......................... Create a new project

 Project Commands:

   build .......................... Build web assets and prepare your app for any platform targets
   capacitor <subcommand> ......... Capacitor functionality (subcommands: add, build, copy, open, run, sync, update)
                                    (alias: cap)
   cordova <subcommand> ........... Cordova functionality (subcommands: build, compile, emulate, platform, plugin,
                                    prepare, requirements, resources, run) (alias: cdv)
   enterprise <subcommand> ........ (paid) Manage Ionic Enterprise features (subcommands: register)
   generate ....................... Create Pages, Components, & Angular Features (alias: g)
   git <subcommand> ............... (paid) Commands relating to managing Appflow git (subcommands: remote)
   integrations <subcommand> ...... Manage various integrations in your app (subcommands: disable, enable, list)
                                    (aliases: i, integration)
   link ........................... (paid) Connect local apps to Ionic
   live-update <subcommand> ....... (paid) Ionic Live Updates functionality (subcommands: add, configure)
   repair ......................... Remove and recreate dependencies and generated files
   serve .......................... Start a local dev server for app dev/testing (alias: s)
   ssl <subcommand> ............... (deprecated) Commands for managing SSL keys & certificates (subcommands: generate)
```

3.2 Creación del proyecto Ionic

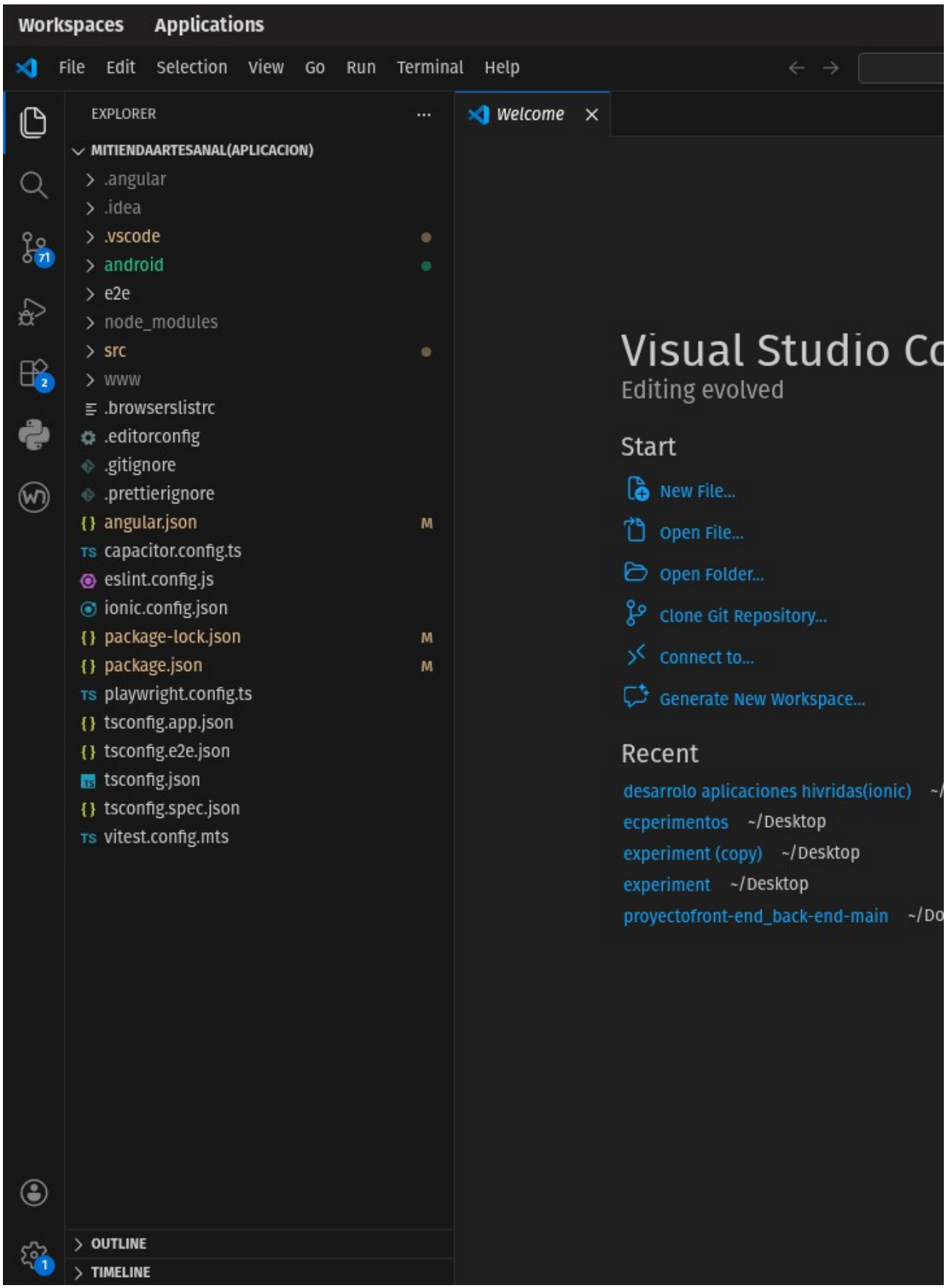
Se creó el proyecto con la plantilla tabs (pestañas), que proporciona una estructura de navegación con múltiples secciones. El comando utilizado fue:

bash

ionic start miTiendaArtesanal(nombre de proyecto) tabs --type=angular --capacitor --standalone

Este comando generó una estructura de carpetas con:

- src/: código fuente de la aplicación Angular.
- android/ e ios/: proyectos nativos (se generan al añadir las plataformas).
- capacitor.config.ts: configuración de Capacitor.
- package.json: dependencias y scripts del proyecto.


Workspaces
Applications
File
Edit
Selection
View
Go
Run
Terminal
Help
EXPLORER
MITIENDAARTESANAL(APLICACION)
.angular
.idea
.vscode
android
e2e
node_modules
src
www
.browserslistrc
.editorconfig
.gitignore
.prettierignore
angular.json
capacitor.config.ts
eslint.config.js
ionic.config.json
package-lock.json
package.json
playwright.config.ts
tsconfig.app.json
tsconfig.e2e.json
tsconfig.json
tsconfig.spec.json
vitest.config.mts
OUTLINE
TIMELINE
Welcome
Visual Studio Co
Editing evolved
Start
New File...
Open File...
Open Folder...
Clone Git Repository...
Connect to...
Generate New Workspace...
Recent
desarrolo aplicaciones hivridas(ionic)
ecperimentos ~/Desktop
experiment (copy) ~/Desktop
experiment ~/Desktop
proyectofront-end_back-end-main

3.3 Instalación de plugins

Se instalaron los plugins de Capacitor necesarios para la cámara, el sistema de archivos y el almacenamiento:

en bash(la terminal de windows(cmd) o la de linux) se escrive:

npm install @capacitor/camera

npm install @capacitor/filesystem

npm install @capacitor/preferences

npm install capacitor-secure-storage-plugin

npm install @ionic/pwa-elements





El plugin @ionic/pwa-elements es necesario para que la cámara funcione en el navegador (emulando un modal con acceso a la webcam), ya que los navegadores no permiten el acceso directo al hardware por parte de aplicaciones web.

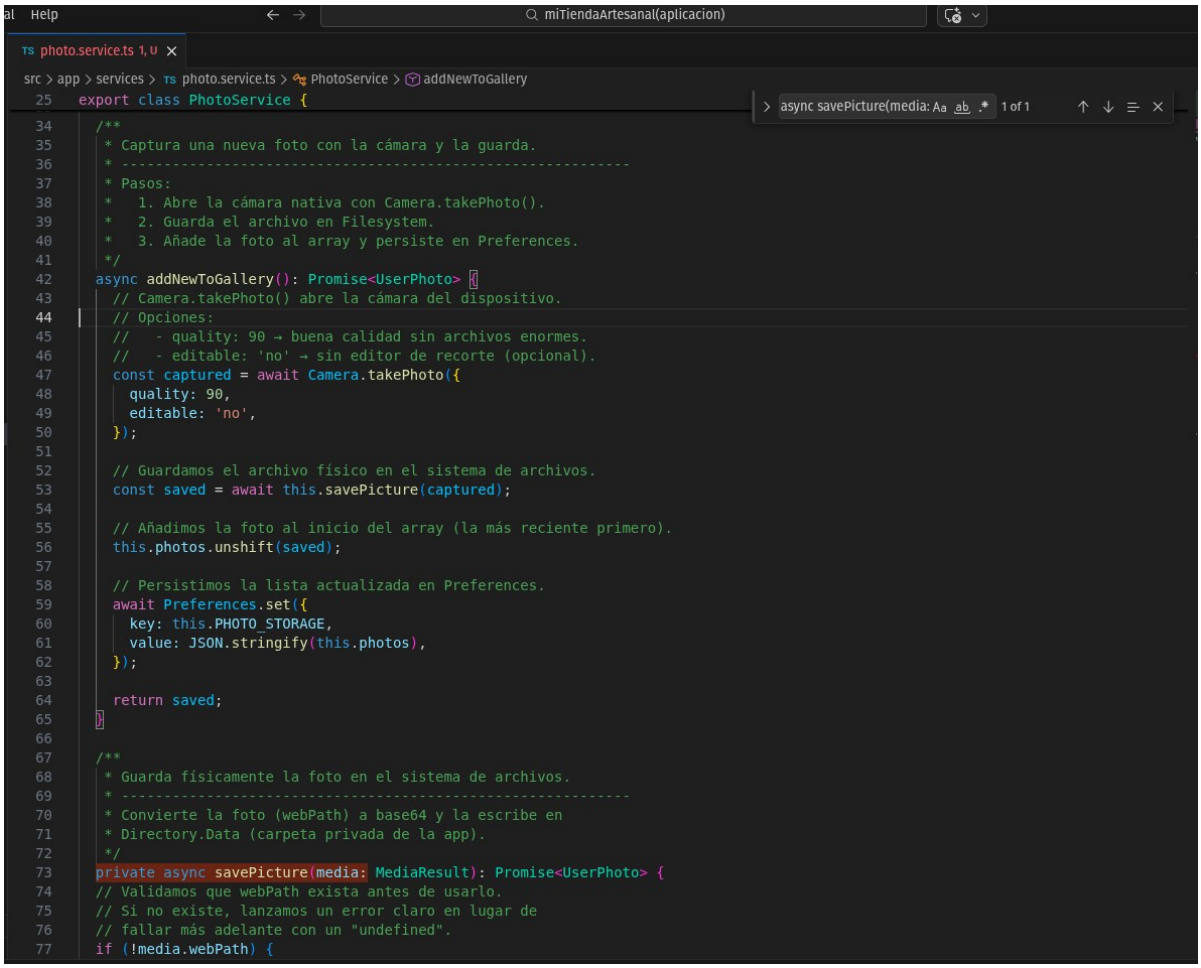
3.4 Creación de los servicios

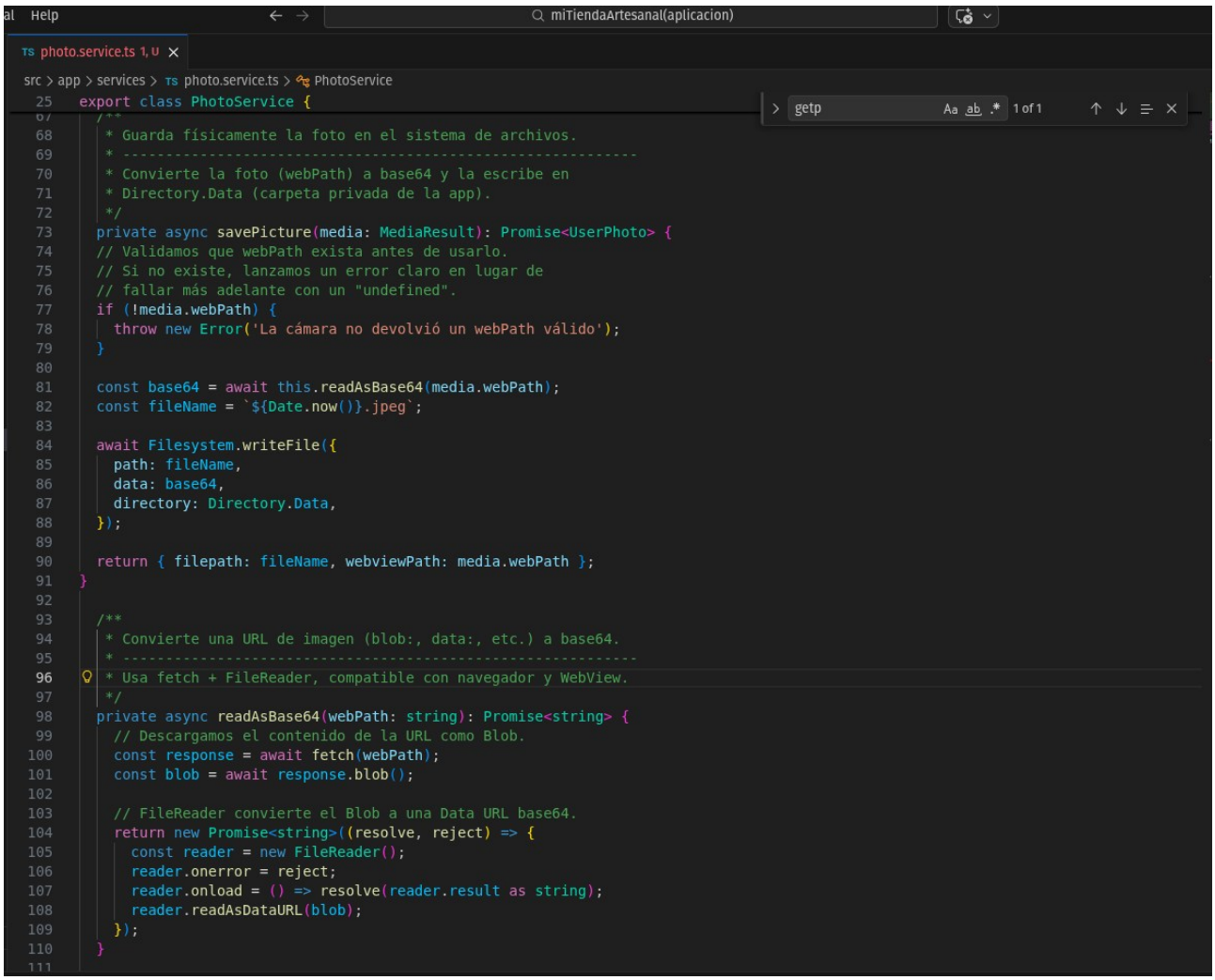
3.4.1 PhotoService

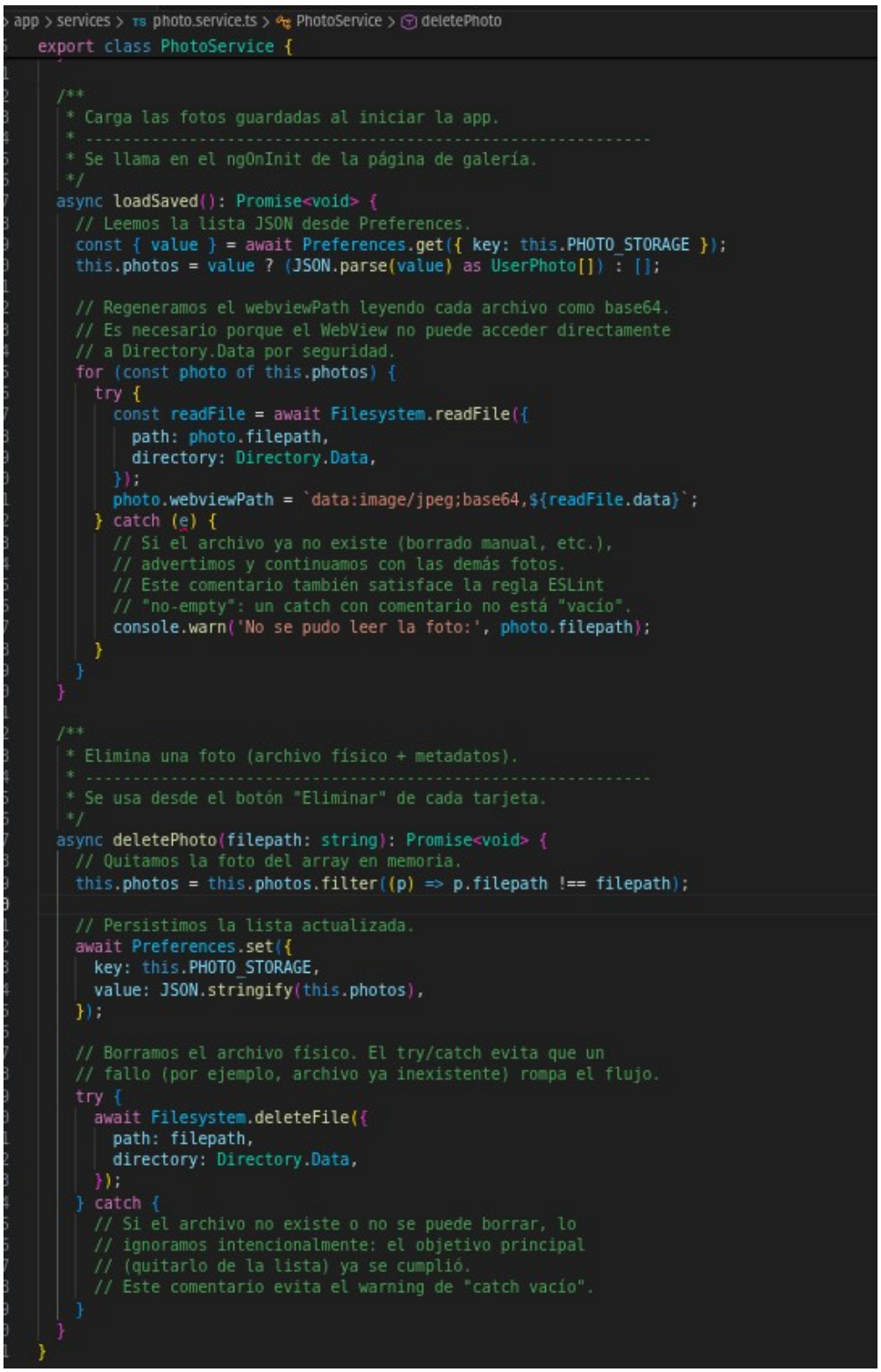
Se creó el servicio PhotoService en src/app/services/photo.service.ts, que encapsula toda la lógica de cámara y almacenamiento. Sus métodos principales son:

- addNewToGallery(): captura una foto con la cámara y la guarda.
- loadSaved(): carga las fotos guardadas al iniciar la app.
- deletePhoto(filepath): elimina una foto del sistema de archivos y de la lista.

El servicio utiliza Camera.TakePhoto() para capturar imágenes, Filesystem.writeFile() para guardarlas como archivos .jpeg en Directory.Data, y Preferences.set() para persistir la lista de metadatos.




Help
miTiendaArtesanal(aplicacion)
photo.service.ts 1, U
src > app > services > photo.service.ts > PhotoService
getp
Aa ab .* 1 of 1
export class PhotoService {
/**
* Guarda físicamente la foto en el sistema de archivos.
* --
* Convierte la foto (webPath) a base64 y la escribe en
* Directory.Data (carpeta privada de la app).
*/
private async savePicture(media: MediaResult): Promise<UserPhoto> {
// Validamos que webPath exista antes de usarlo.
// Si no existe, lanzamos un error claro en lugar de
// fallar más adelante con un "undefined".
if (!media.webPath) {
throw new Error('La cámara no devolvió un webPath válido');
}
const base64 = await this.readAsBase64(media.webPath);
const fileName = `${Date.now()}.jpeg`;
await Filesystem.writeFile({
path: fileName,
data: base64,
directory: Directory.Data,
});
return { filepath: fileName, webviewPath: media.webPath };
}
/**
* Convierte una URL de imagen (blob:, data:, etc.) a base64.
* --
* Usa fetch + FileReader, compatible con navegador y WebView.
*/
private async readAsBase64(webPath: string): Promise<string> {
// Descargamos el contenido de la URL como Blob.
const response = await fetch(webPath);
const blob = await response.blob();
// FileReader convierte el Blob a una Data URL base64.
return new Promise<string>((resolve, reject) => {
const reader = new FileReader();
reader.onerror = reject;
reader.onload = () => resolve(reader.result as string);
reader.readAsDataURL(blob);
});
}

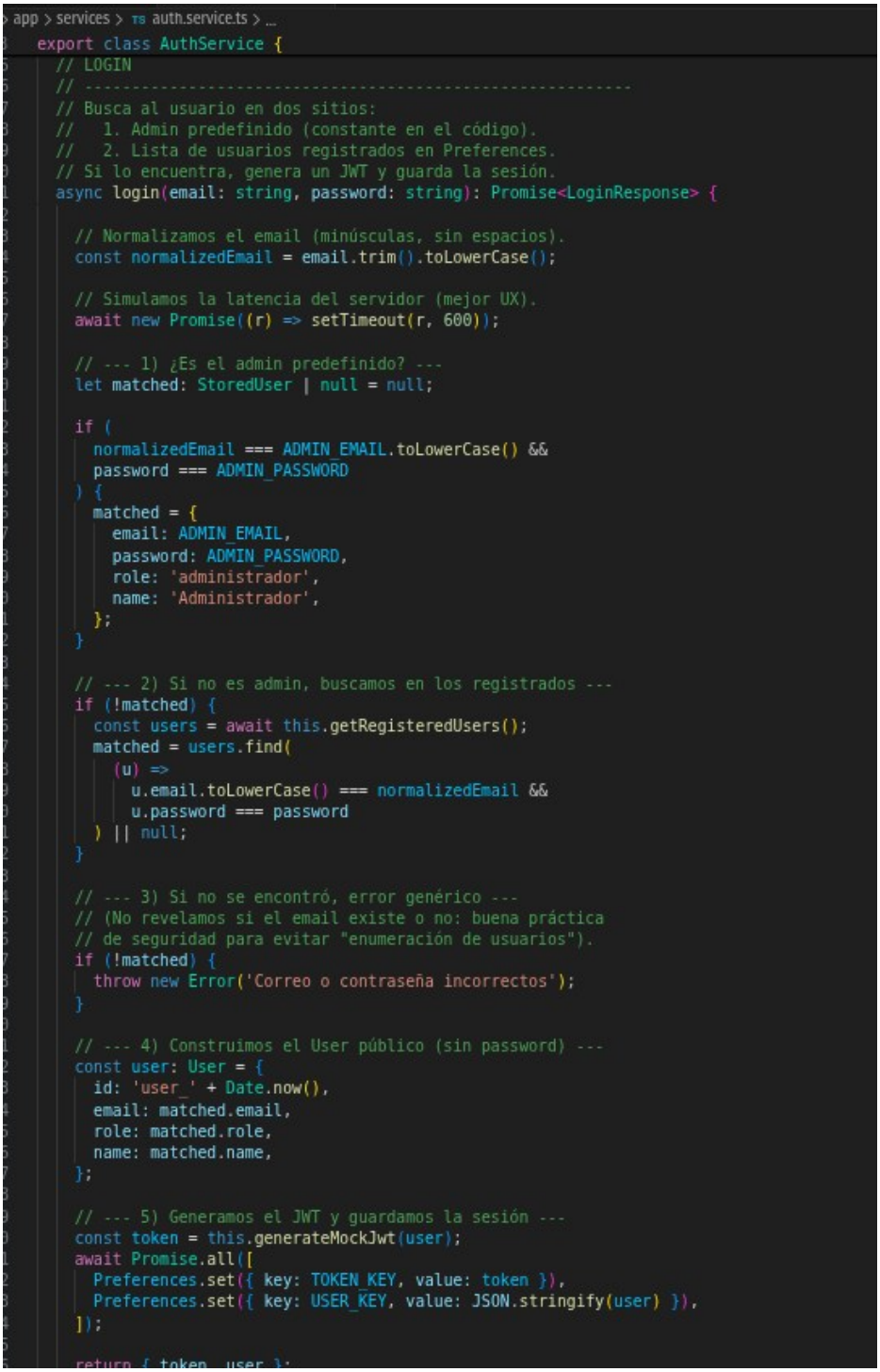


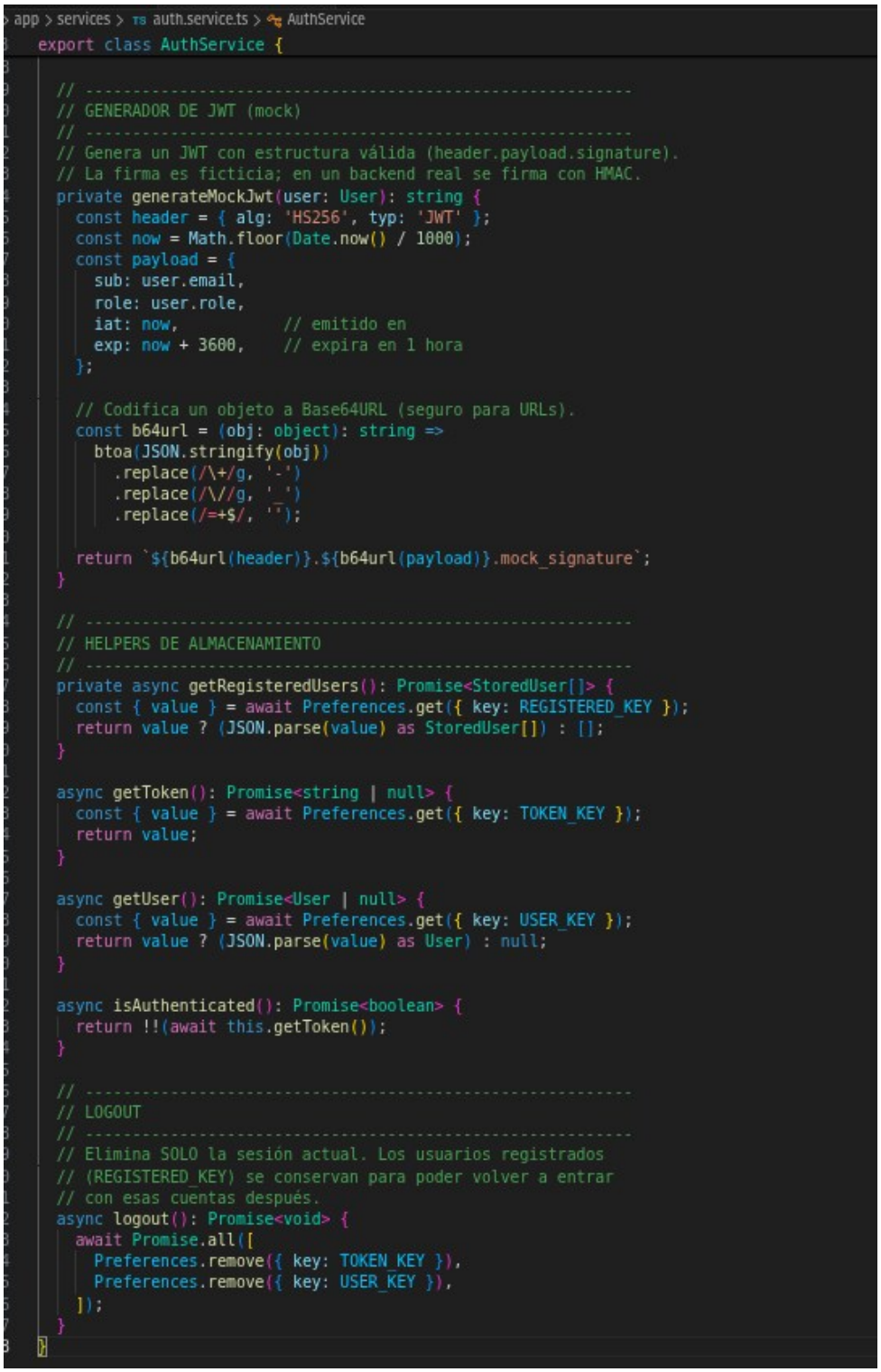
3.4.2 AuthService

Se creó el servicio AuthService en src/app/services/auth.service.ts, que implementa el flujo de autenticación. Sus métodos principales son:

- login(email, password): valida credenciales y genera un JWT.
- register(email, password, role): registra un nuevo usuario.
- logout(): elimina el token y el usuario del almacenamiento.
- getToken(), getUser(), isAuthenticated(): helpers para consultar la sesión.

El administrador (admin@test.com / 123456) está predefinido en el código como constante, mientras que los usuarios compradores y vendedores se almacenan en Preferences bajo la clave registered_users.





```
app > services > auth.service.ts > AuthService
export class AuthService {

  // ------------------------------------------------------------
  // GENERADOR DE JWT (mock)
  // ------------------------------------------------------------
  // Genera un JWT con estructura válida (header.payload.signature).
  // La firma es ficticia; en un backend real se firma con HMAC.
  private generateMockJwt(user: User): string {
    const header = { alg: 'HS256', typ: 'JWT' };
    const now = Math.floor(Date.now() / 1000);
    const payload = {
      sub: user.email,
      role: user.role,
      iat: now,           // emitido en
      exp: now + 3600,    // expira en 1 hora
    };

    // Codifica un objeto a Base64URL (seguro para URLs).
    const b64url = (obj: object): string =>
      btoa(JSON.stringify(obj))
        .replace(/\+/g, '-')
        .replace(/\//g, '_')
        .replace(/=+$/, '');

    return `${b64url(header)}.${b64url(payload)}.mock_signature`;
  }

  // ------------------------------------------------------------
  // HELPERS DE ALMACENAMIENTO
  // ------------------------------------------------------------
  private async getRegisteredUsers(): Promise<StoredUser[]> {
    const { value } = await Preferences.get({ key: REGISTERED_KEY });
    return value ? (JSON.parse(value) as StoredUser[]) : [];
  }

  async getToken(): Promise<string | null> {
    const { value } = await Preferences.get({ key: TOKEN_KEY });
    return value;
  }

  async getUser(): Promise<User | null> {
    const { value } = await Preferences.get({ key: USER_KEY });
    return value ? (JSON.parse(value) as User) : null;
  }

  async isAuthenticated(): Promise<boolean> {
    return !!(await this.getToken());
  }

  // ------------------------------------------------------------
  // LOGOUT
  // ------------------------------------------------------------
  // Elimina SOLO la sesión actual. Los usuarios registrados
  // (REGISTERED_KEY) se conservan para poder volver a entrar
  // con esas cuentas después.
  async logout(): Promise<void> {
    await Promise.all([
      Preferences.remove({ key: TOKEN_KEY }),
      Preferences.remove({ key: USER_KEY }),
    ]);
  }
}
```

3.4.3 AuthInterceptor

Se creó el interceptor funcional authInterceptor en src/app/interceptors/auth.interceptor.ts, que intercepta cada petición HTTP y adjunta el token JWT en la cabecera Authorization si existe una sesión activa.

app > interceptors > TS auth.interceptor.ts > ...

```ts
// ============================================================
// INTERCEPTOR DE AUTENTICACIÓN
// ------------------------------------------------------------
// Un "interceptor" en Angular es una función que se ejecuta
// ANTES de que cualquier petición HTTP salga hacia el backend.
// Aquí aprovechamos ese momento para:
//   - Leer el token JWT guardado en Preferences.
//   - Si existe, clonar la petición y añadir la cabecera
//     "Authorization: Bearer <token>".
//   - Si no existe, dejar pasar la petición tal cual.
//
// Corresponde al requerimiento RF-02 (gestión de tokens de sesión).
// ============================================================

import type { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { from, switchMap } from 'rxjs';

import { AuthService } from '../services/auth.service';



/**
 * Interceptor funcional (estilo moderno de Angular 15+).
 *
 * Recibe:
 *   - req: la petición HTTP original que el usuario quiere hacer.
 *   - next: la función que "continúa" el flujo hacia el servidor.
 *
 * Devuelve: un Observable con la respuesta del servidor.
 */
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  // inject() es la forma moderna de obtener dependencias dentro
  // de una función (en vez de usar constructor). En un interceptor
  // funcional es la única forma de acceder al AuthService.
  const authService = inject(AuthService);

  // from(...) convierte la Promise de getToken() en un Observable
  // para poder encadenarlo con el resto del flujo reactivo.
  return from(authService.getToken()).pipe(
    // switchMap permite esperar el token y luego continuar
    // con la petición HTTP ya modificada.
    switchMap((token) => {
      if (token) {
        // req.clone() es OBLIGATORIO porque las peticiones HTTP
        // en Angular son INMUTABLES: no se pueden modificar
        // directamente, hay que clonarlas.
        const cloned = req.clone({
          // Añadimos (o sobreescribimos) la cabecera Authorization
          // con el formato estándar "Bearer <token>".
          headers: req.headers.set('Authorization', `Bearer ${token}`),
        });
        // Enviamos la petición clonada al servidor.
        return next(cloned);
      }
      // Si no hay token, dejamos pasar la petición original
      // (por ejemplo, para endpoints públicos como /login).
      return next(req);
    })
  );
};
```

3.4.4 AuthGuard

Se creó el guard authGuard en src/app/guards/auth.guard.ts, que protege la ruta /tabs y redirige a /login si no hay token guardado.

src > app > guards > TS auth.guard.ts > ...

```ts
// ============================================================
// GUARD DE AUTENTICACIÓN (authGuard)
// ------------------------------------------------------------
// Un "guard" (guardián) es una función que Angular ejecuta
// ANTES de activar una ruta. Sirve para decidir si el usuario
// puede entrar o no.
//
// Este guard:
//   - Si el usuario tiene token → permite el acceso (true).
//   - Si NO tiene token → redirige a /login (UrlTree).
//
// Se aplica a la ruta /tabs para proteger toda la app.
// ============================================================

import { inject } from '@angular/core';
import { Router } from '@angular/router';
import type { CanActivateFn } from '@angular/router';

import { AuthService } from '../services/auth.service';

// "CanActivateFn" es el TIPO de función de guard moderno
// (Angular 15+). Reemplaza a las clases que implementaban
// CanActivate.
export const authGuard: CanActivateFn = async () => {

  // inject() obtiene dependencias dentro de una función
  // (no podemos usar constructor porque no es una clase).
  const authService = inject(AuthService);
  const router = inject(Router);

  // Consultamos si hay token guardado.
  const isAuth = await authService.isAuthenticated();

  // Si está autenticado, permitimos la navegación.
  if (isAuth) return true;

  // Si no, devolvemos una UrlTree que redirige a /login.
  // Angular interpreta el UrlTree como "cancela la navegación
  // actual y ve a esta otra ruta".
  return router.createUrlTree(['/login']);
};
```

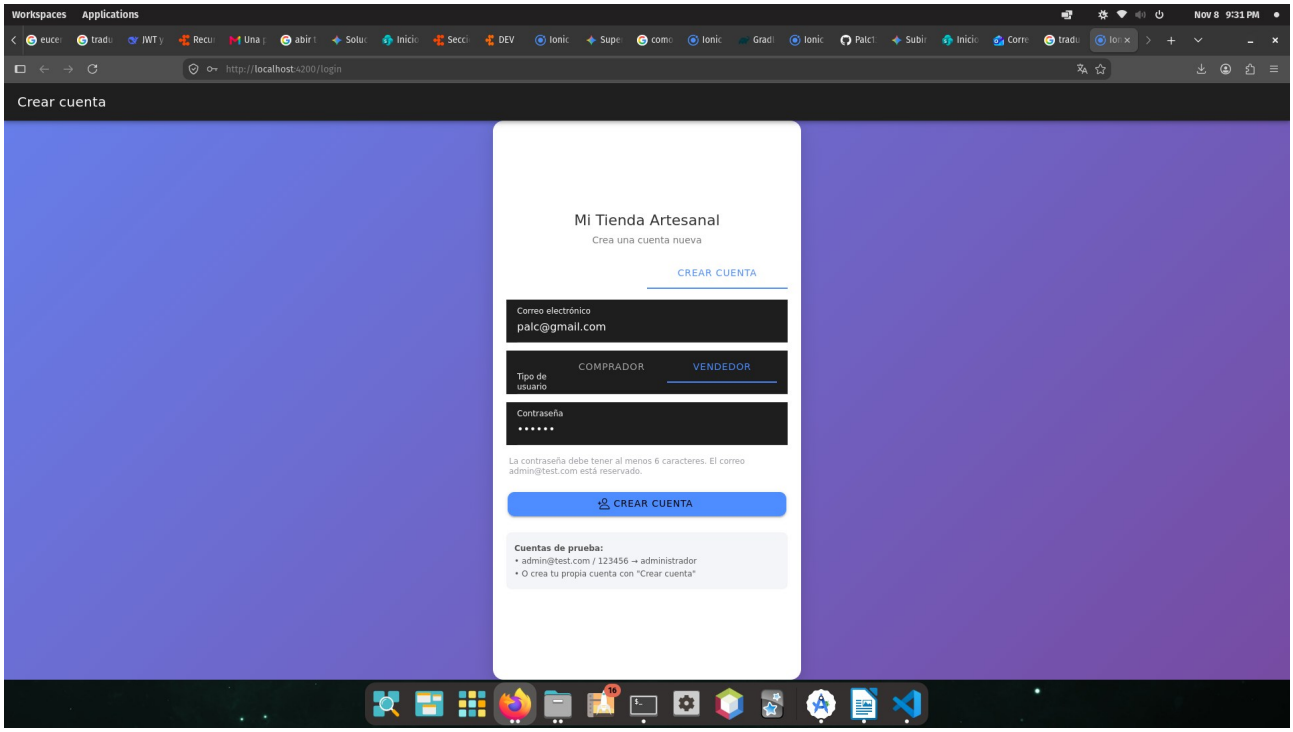
3.5 Interfaz de usuario

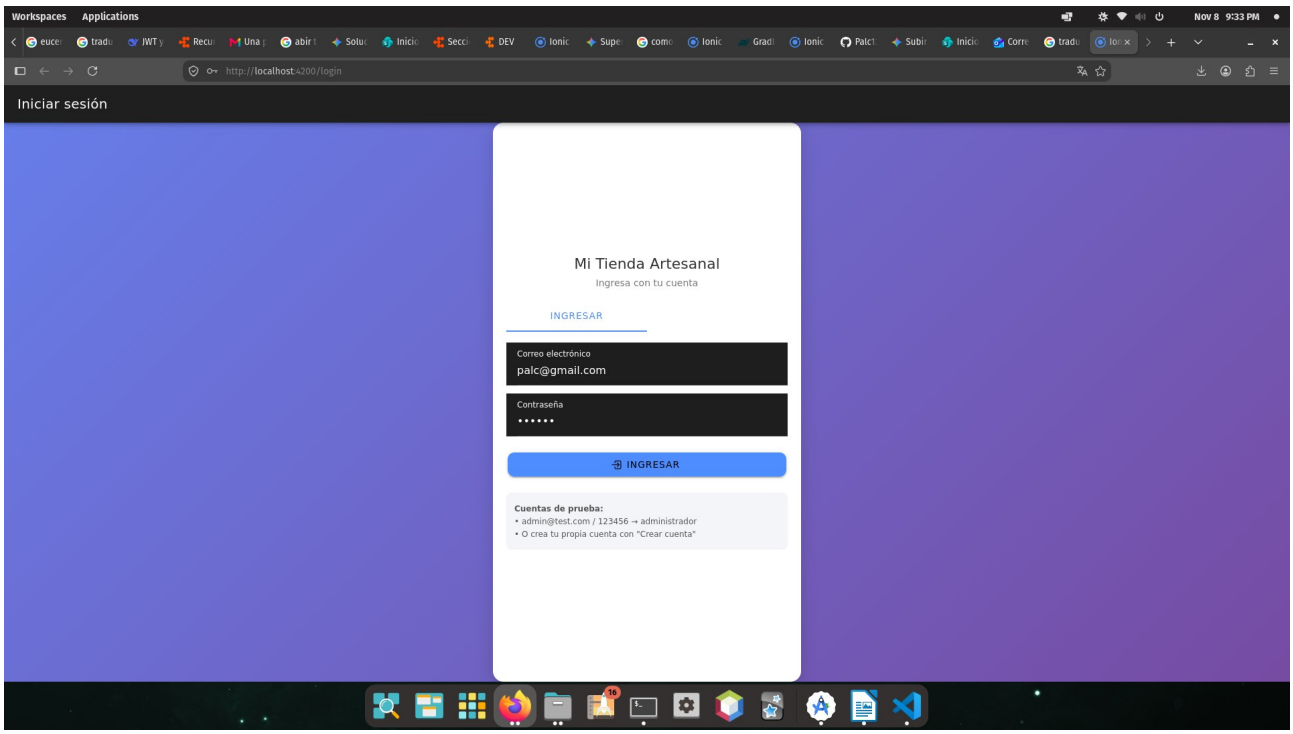
3.5.1 Página de Login/Registro

Se construyó la página LoginPage en src/app/pages/login/, con dos modos controlados por un ion-segment:

- Modo Login: formulario con email y contraseña.

- Modo Registro: formulario con email, selector de rol (comprador o vendedor) y contraseña.

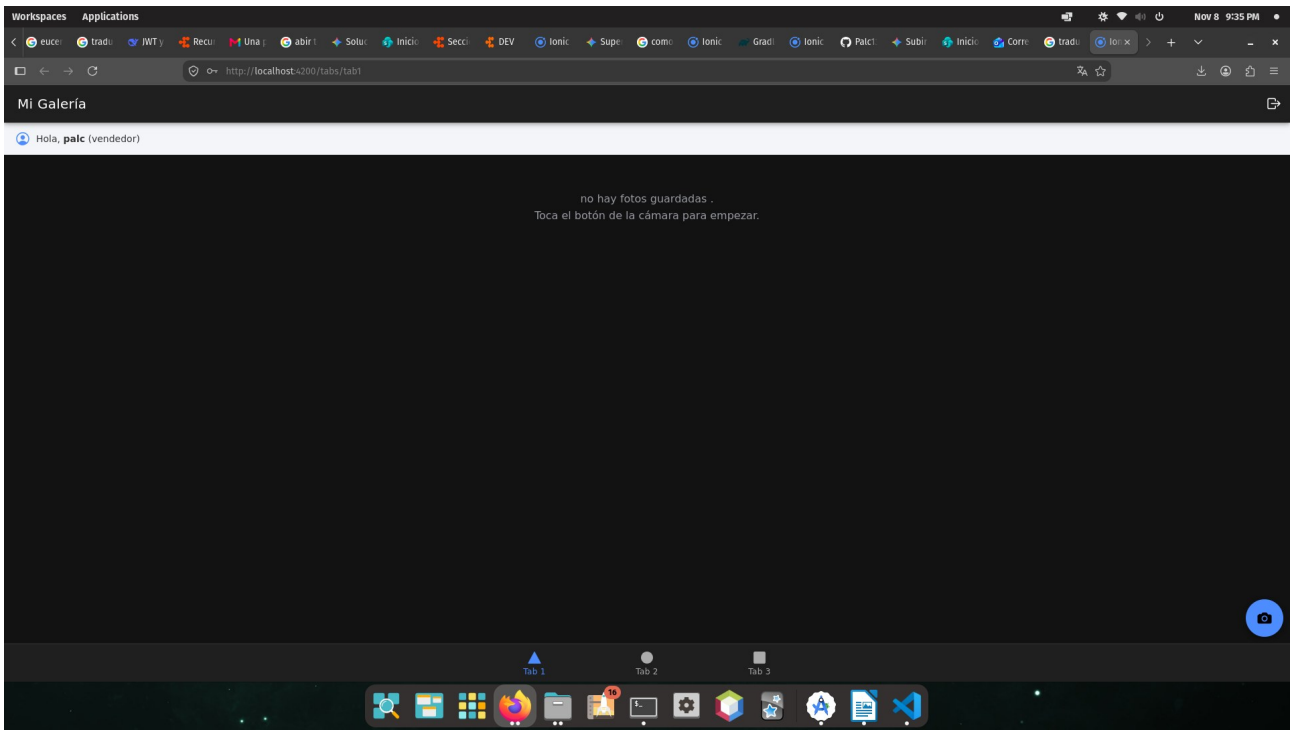


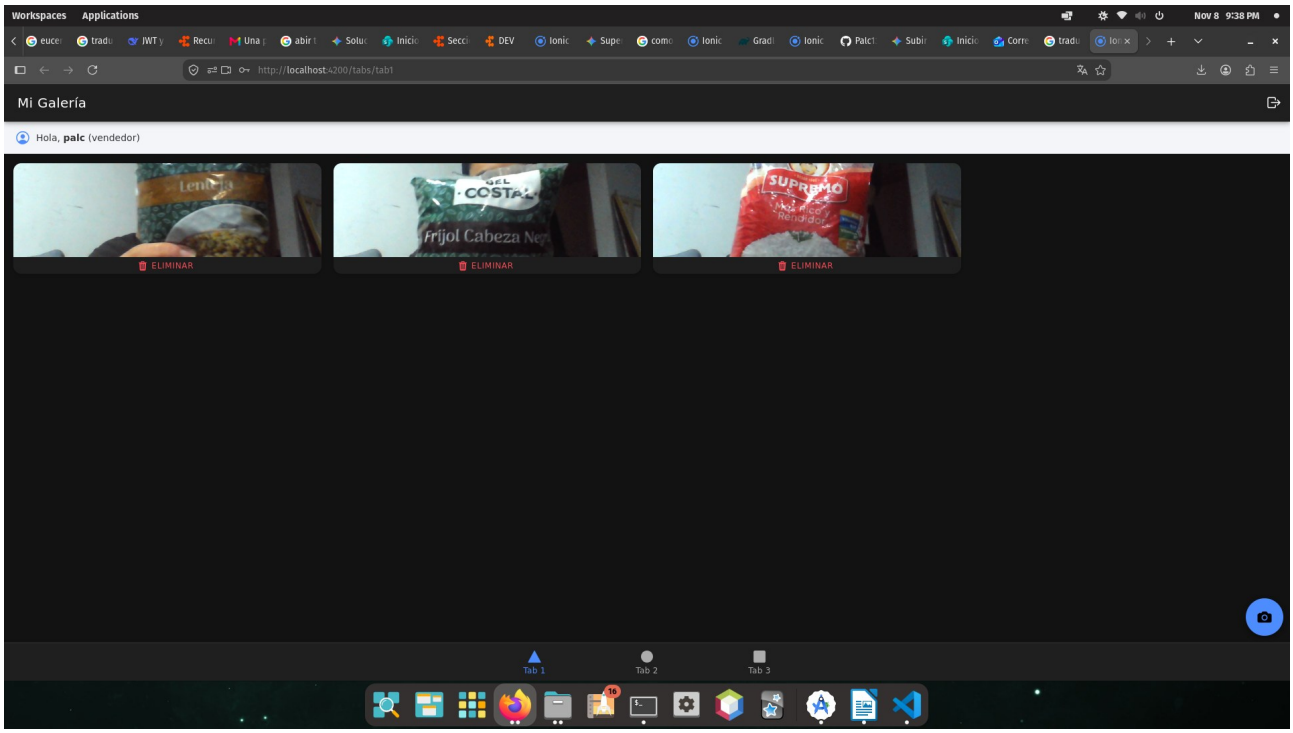


3.5.2 Página de Galería

Se modificó la página Tab1Page para mostrar:

- Un saludo al usuario autenticado ("Hola, {nombre} ({rol})").
- Un botón de logout en el header (ícono de puerta saliendo).
- Un grid responsivo con las fotos capturadas.
- Un botón flotante (FAB) para tomar nuevas fotos.
- Un mensaje vacío cuando no hay fotos.





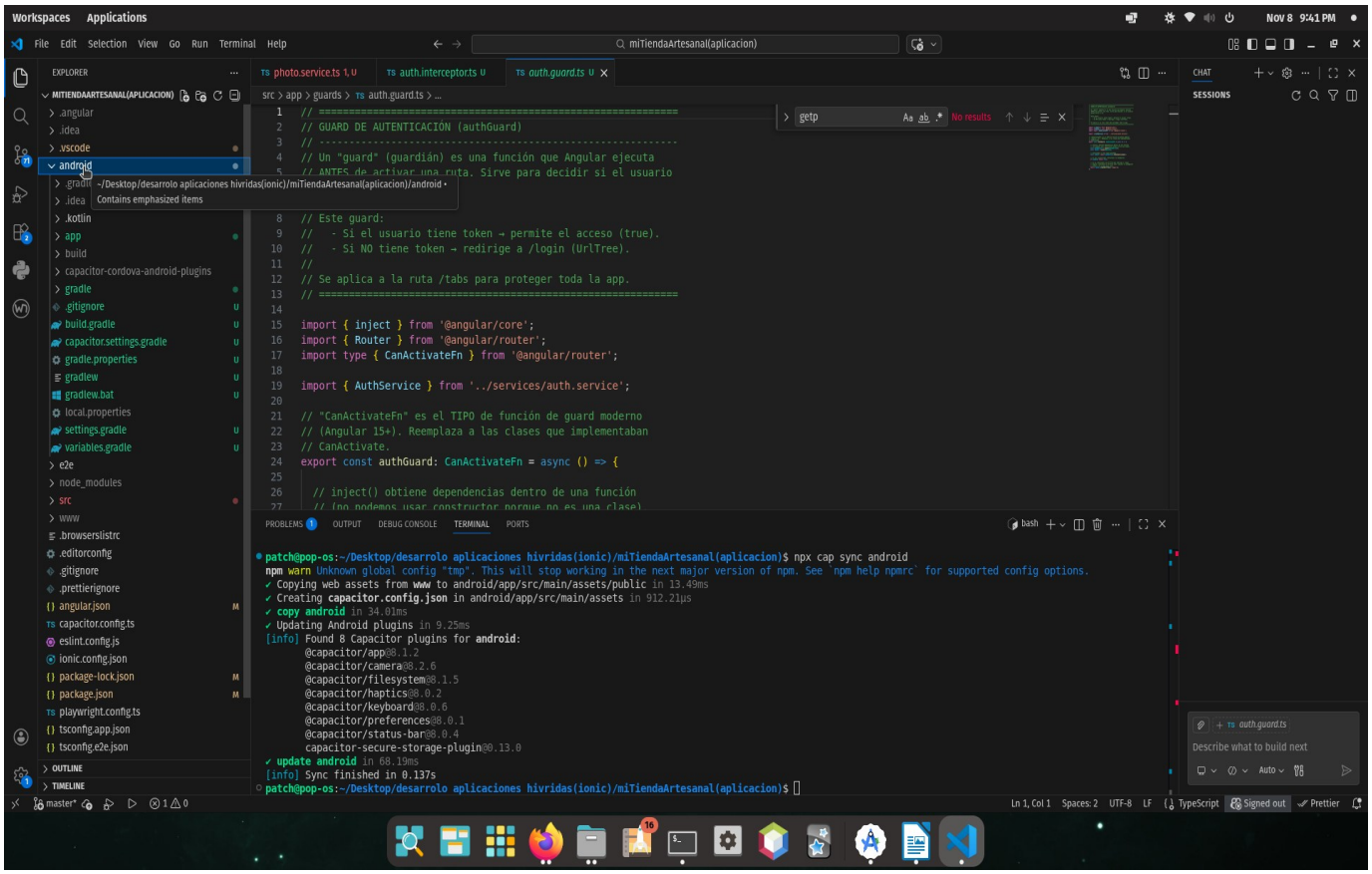
3.6 Migración a Android

Una vez implementada y probada la versión web, se procedió a la migración a Android. Los comandos utilizados fueron:

npm install @capacitor/android

npx cap add android

npx cap sync android

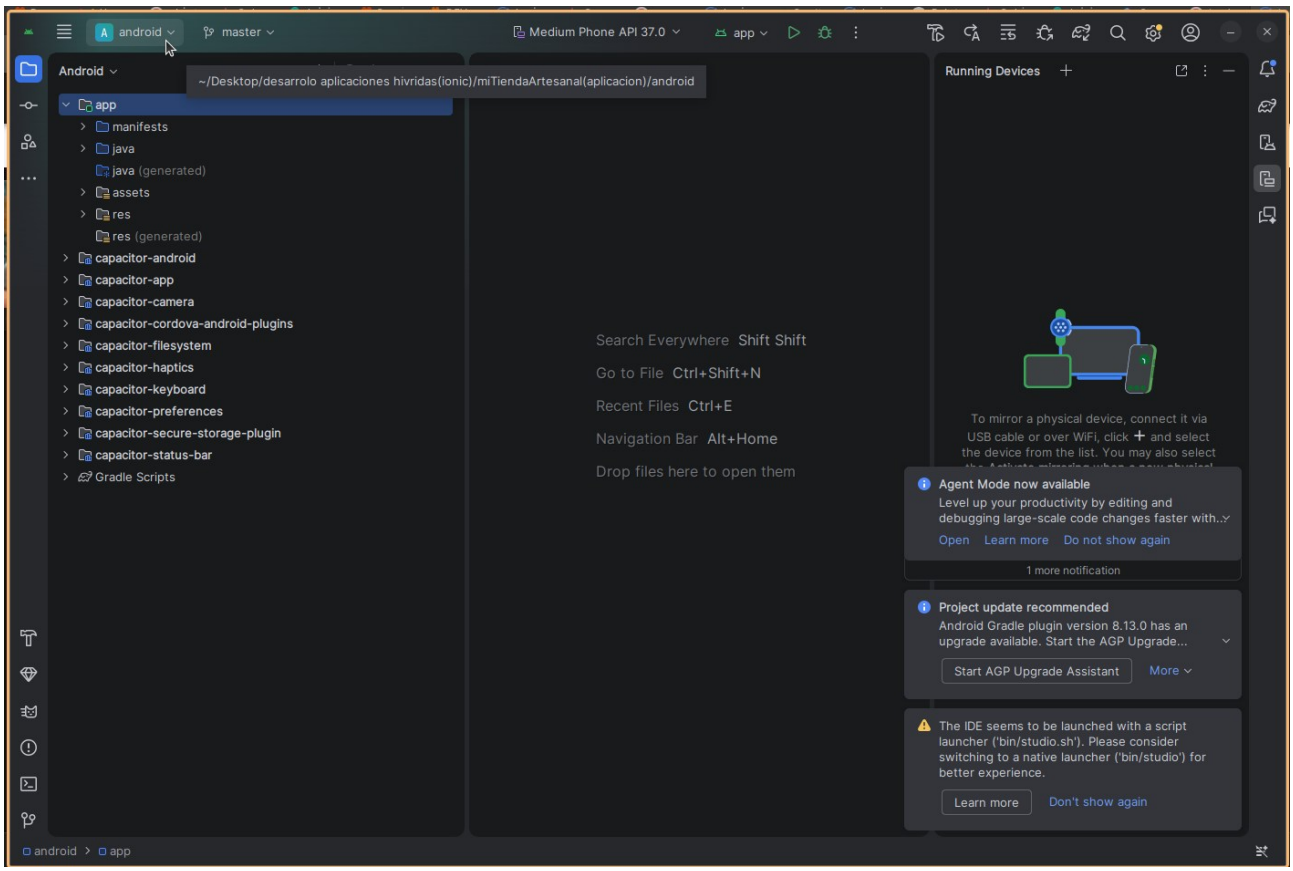


Esto generó la carpeta android/ con un proyecto Android Studio completo.

Posteriormente se abrió el proyecto en Android Studio:

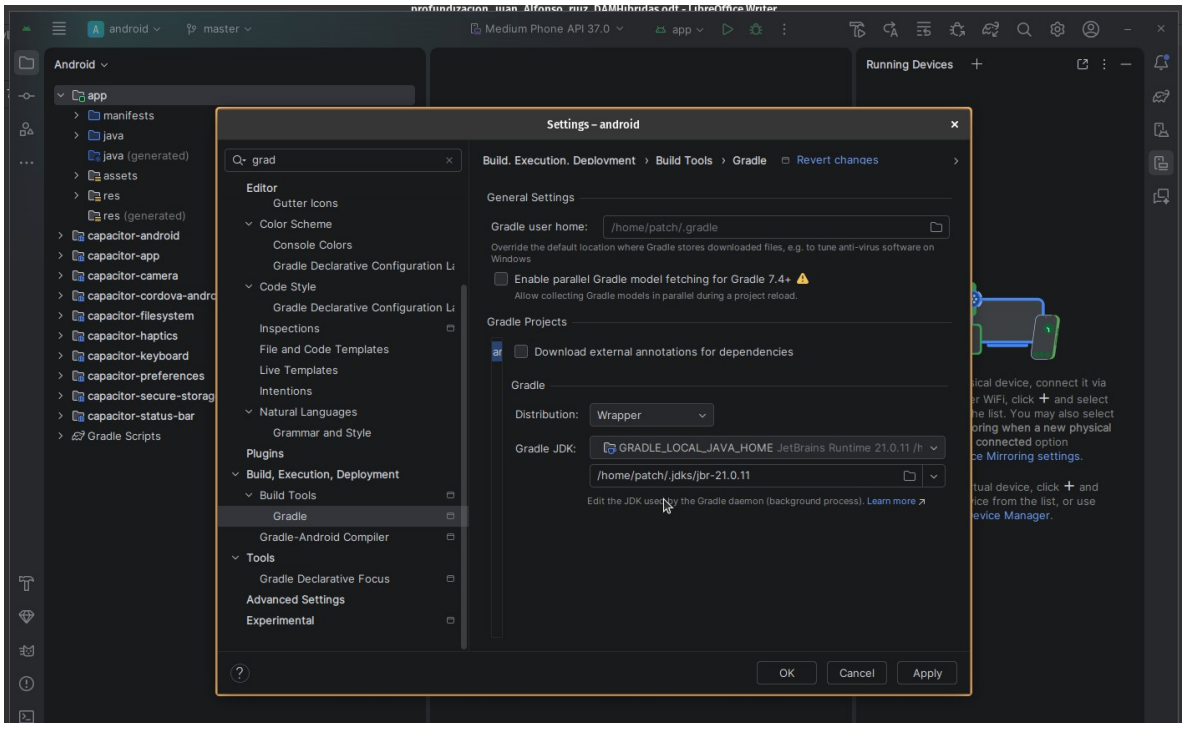
npx cap open android

Durante la primera apertura, Android Studio sincronizó Gradle (el sistema de compilación de Android), descargando las dependencias necesarias.



3.7 Ajuste de la versión de Java

Android Studio mostró inicialmente un error de incompatibilidad entre Gradle 8.14.3 y Java 25. Se resolvió cambiando el "Gradle JDK" a JDK 21 desde File → Settings → Build, Execution, Deployment → Build Tools → Gradle.



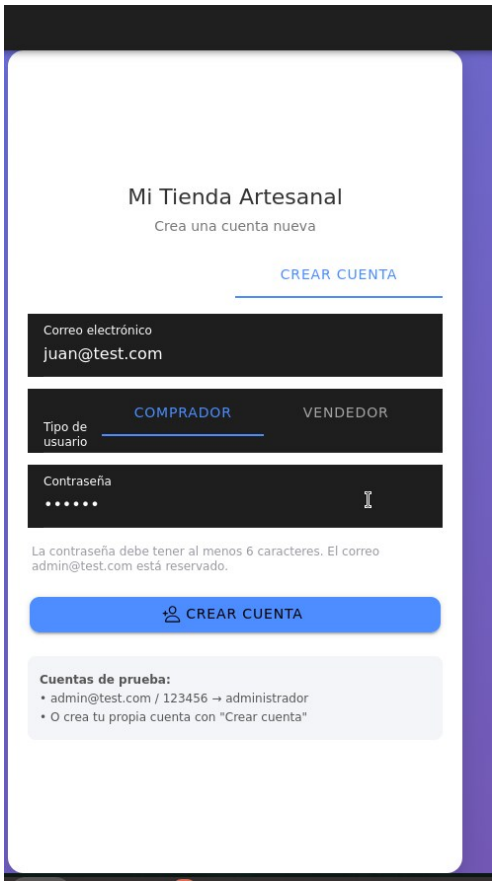
4. PRUEBAS

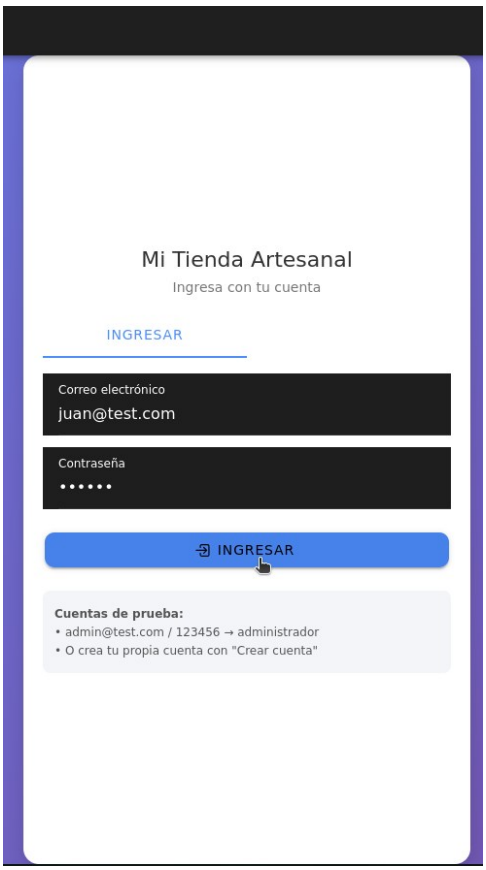
4.1 Pruebas en el navegador

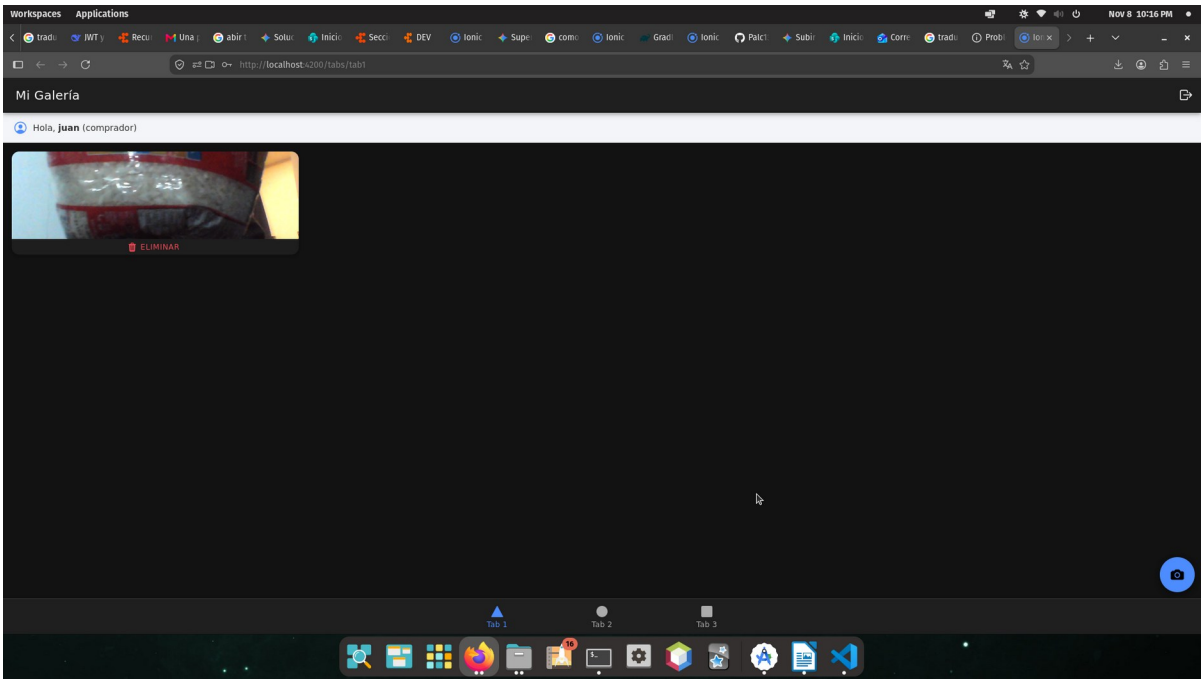
Se ejecutó la aplicación en el navegador con ionic serve (visual studio habre el proyecto solo si se ejecuta el comando en la linea de comandos de vs code), accediendo a http://localhost:8100 . Se realizaron las siguientes pruebas:

4.1.1 Prueba del flujo de autenticación

- Al abrir la aplicación sin sesión, el authGuard redirigió automáticamente a /login.
- Se registró una cuenta de prueba (juan@test.com / 123456, rol comprador).
- Se verificó el login automático tras el registro.
- Se verificó que el usuario aparecía en el
- header ("Hola, juan (comprador)").
- Se probó el logout y la redirección al login.



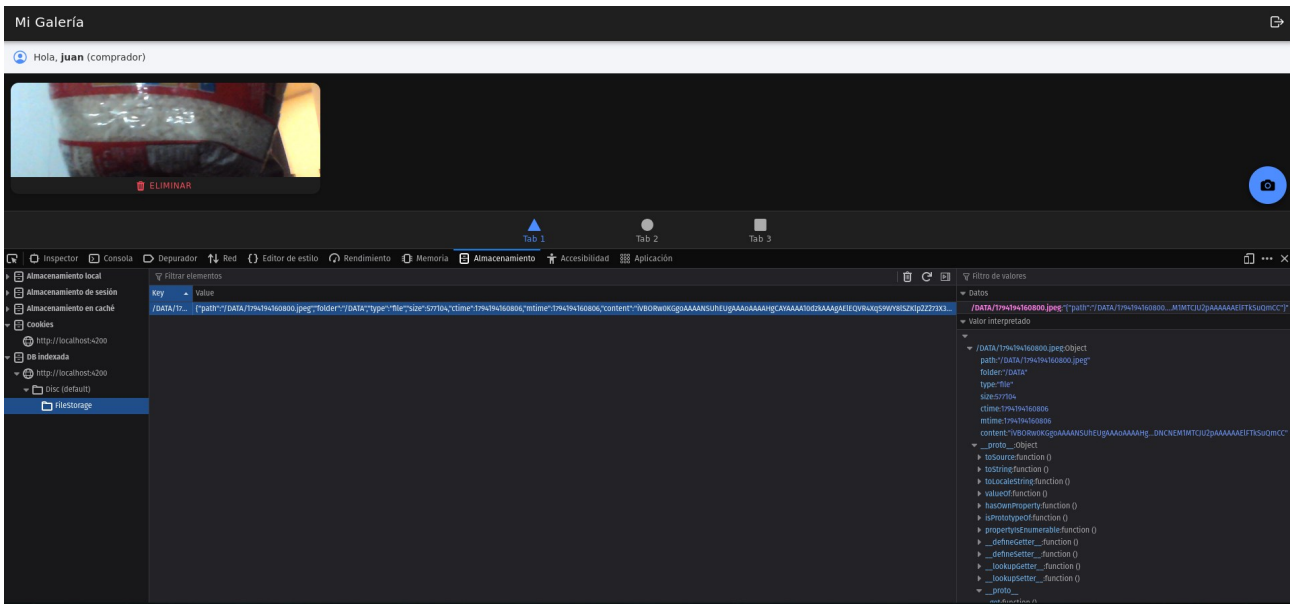


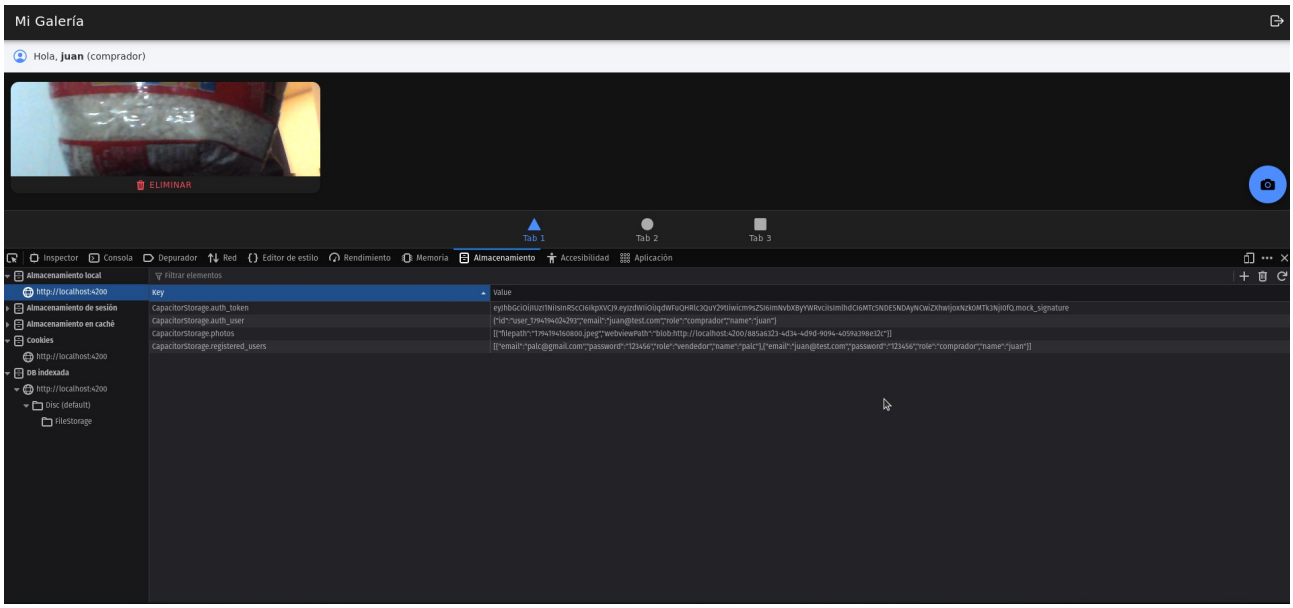


4.1.2 Verificación del almacenamiento

Se abrieron las DevTools del navegador (F12 (o click derecho, inspeccionar)→ Application luego: IndexedDB --- _ionicstorage--- keyvalue) y se verificó la presencia de:

- auth_token: el JWT de la sesión (formato xxxxx.yyyyy.mock_signature).
- auth_user: el JSON del usuario autenticado.
- registered_users: la lista de usuarios registrados.

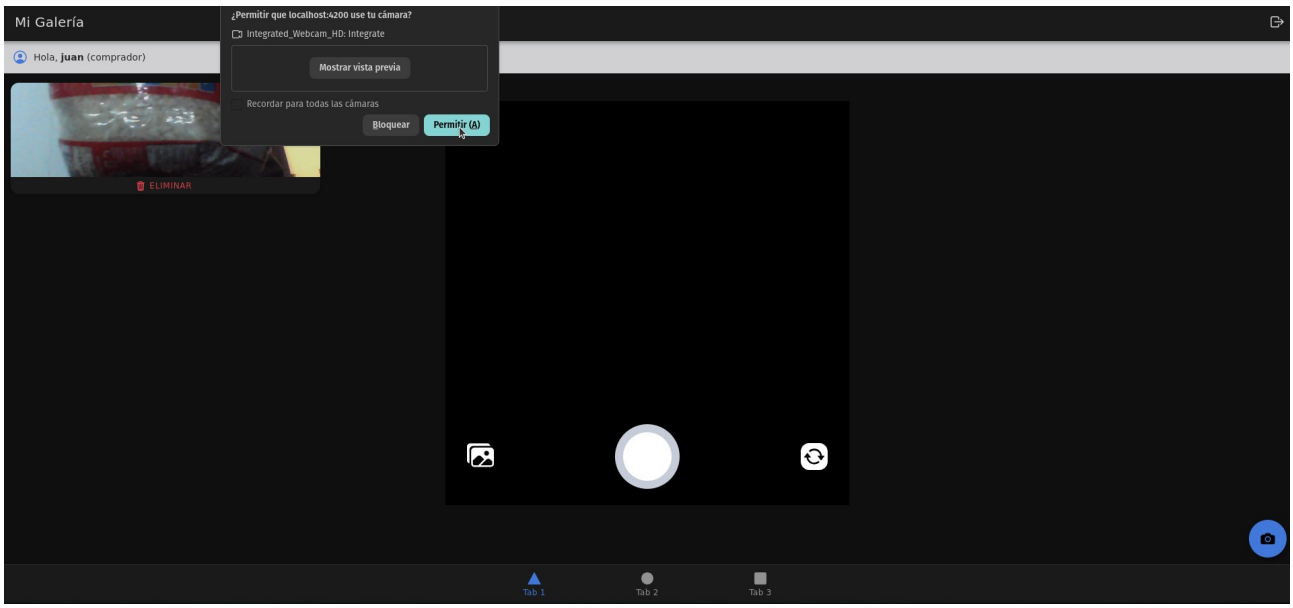


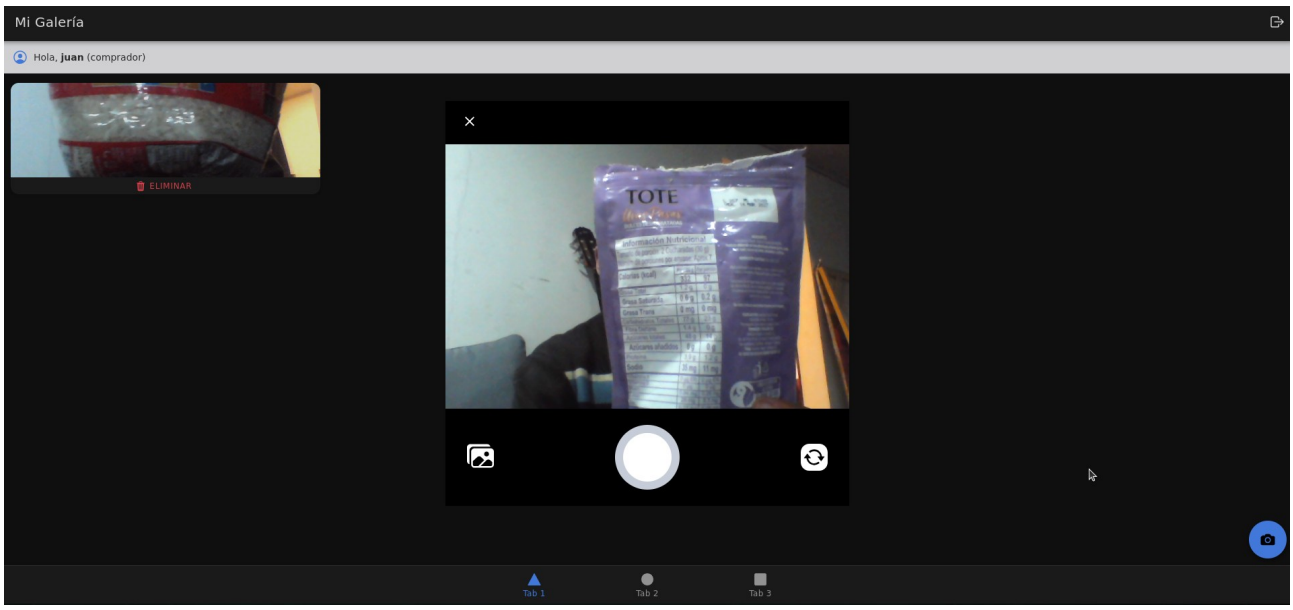


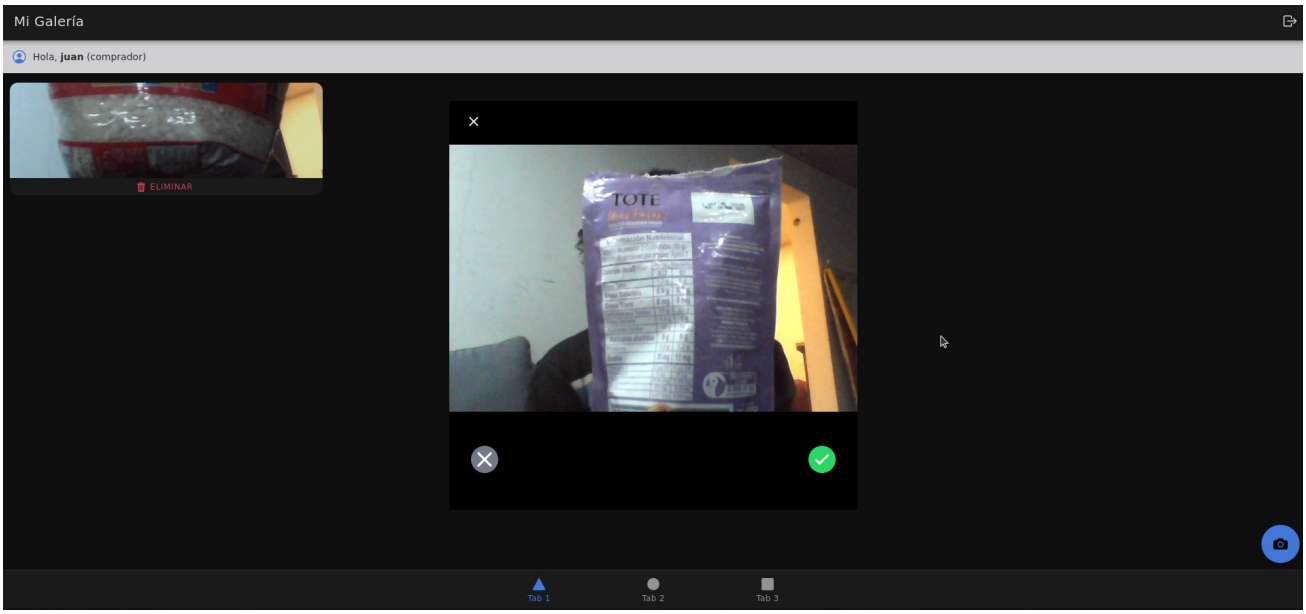
4.1.3 Prueba de la galería

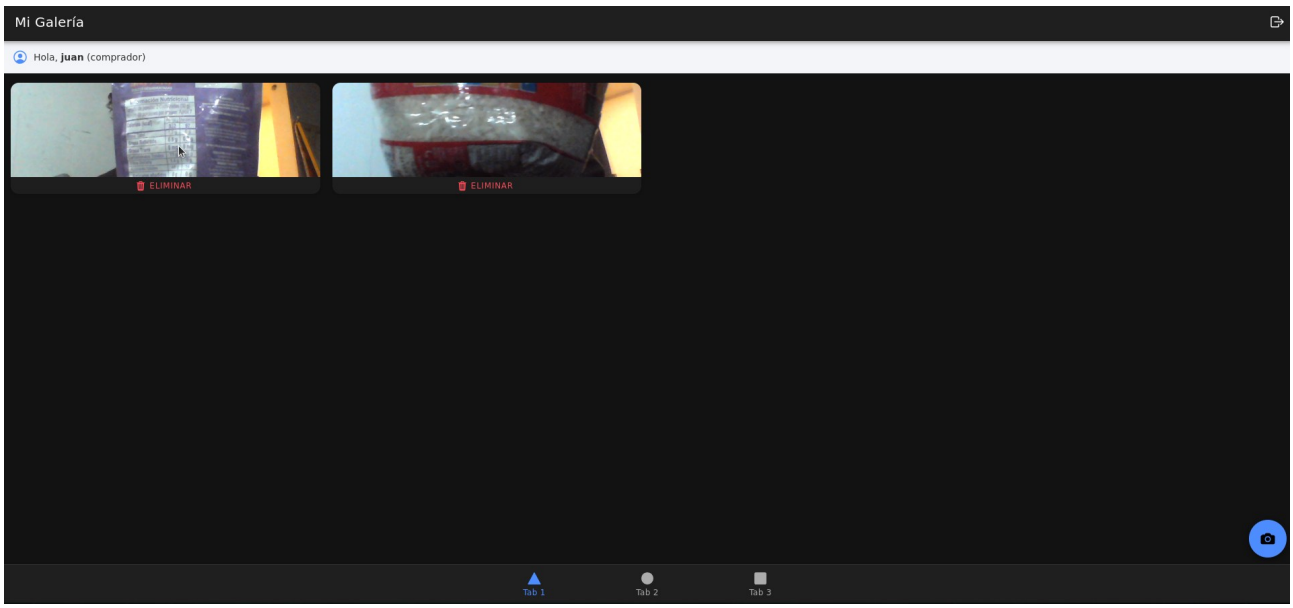
- Se hizo clic en el botón flotante de cámara.
- Se aceptó el permiso de la webcam en el navegador.
- Se capturó una foto en el modal de PWA Elements.
- La foto apareció inmediatamente en el grid (gracias al ChangeDetectorRef.detectChanges()).
- Se recargó la página (F5) y la foto persistió.

- Se probó la eliminación de una foto y se verificó la desaparición inmediata.


Mi Galería
¿Permitir que localhost:4200 use tu cámara?
Integrated_Webcam_HD: Integrate
Mostrar vista previa
Recordar para todas las cámaras
Bloquear
Permitir (A)
Hola, juan (comprador)
ELIMINAR
Tab 1
Tab 2
Tab 3

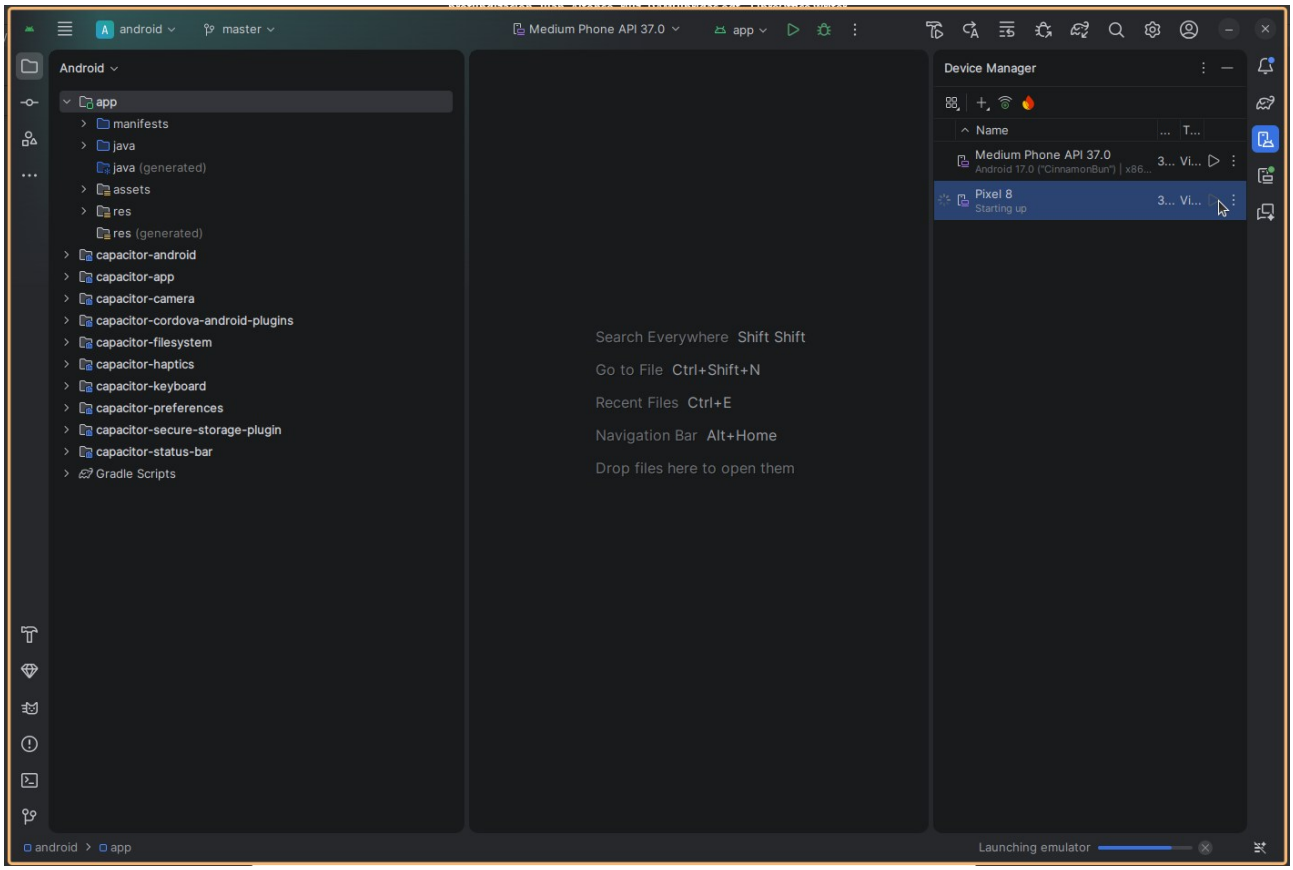

Mi Galería
Hola, juan (comprador)
ELIMINAR
TOTE
Tab 1
Tab 2
Tab 3

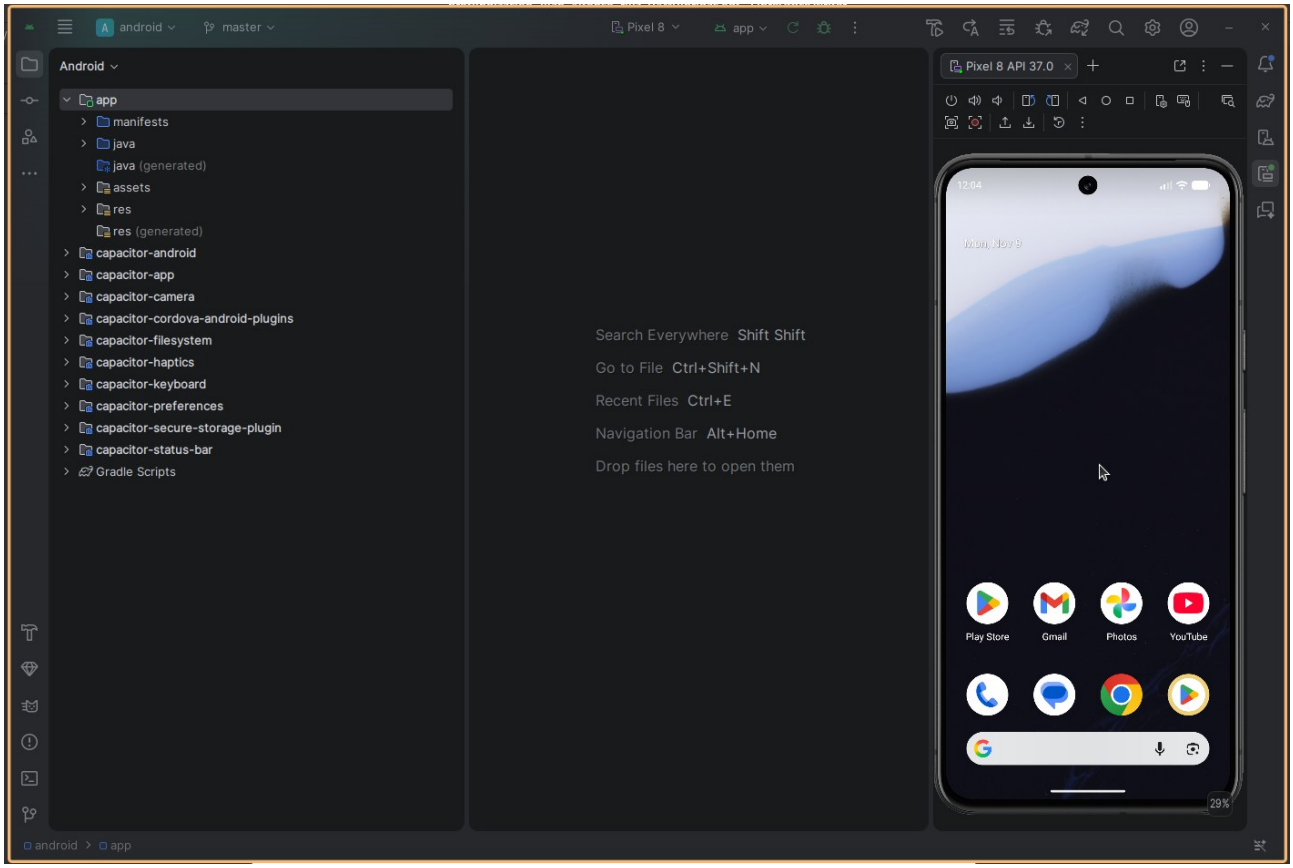

Mi Galería
Hola, juan (comprador)
ELIMINAR
TOTE
Tab 1
Tab 2
Tab 3



4.2 Pruebas en el emulador Android

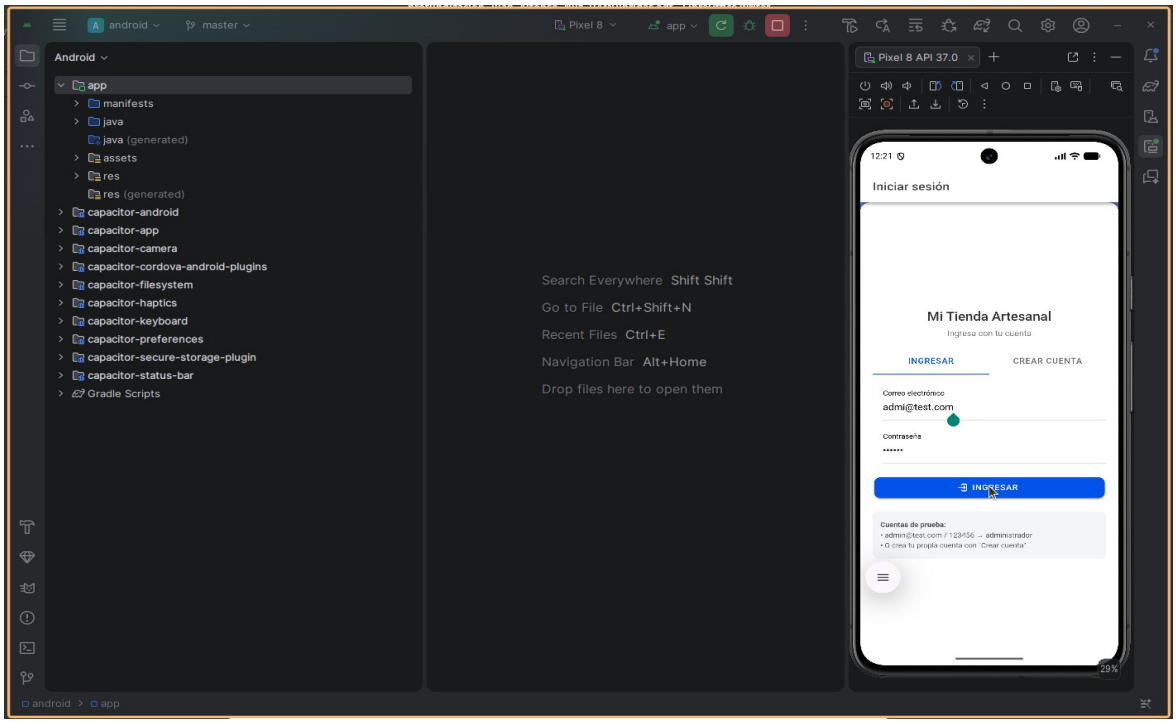
Se creó un emulador (AVD) con Android 14 (API 37, pixel 8) en Android Studio y se ejecutó la aplicación en él.





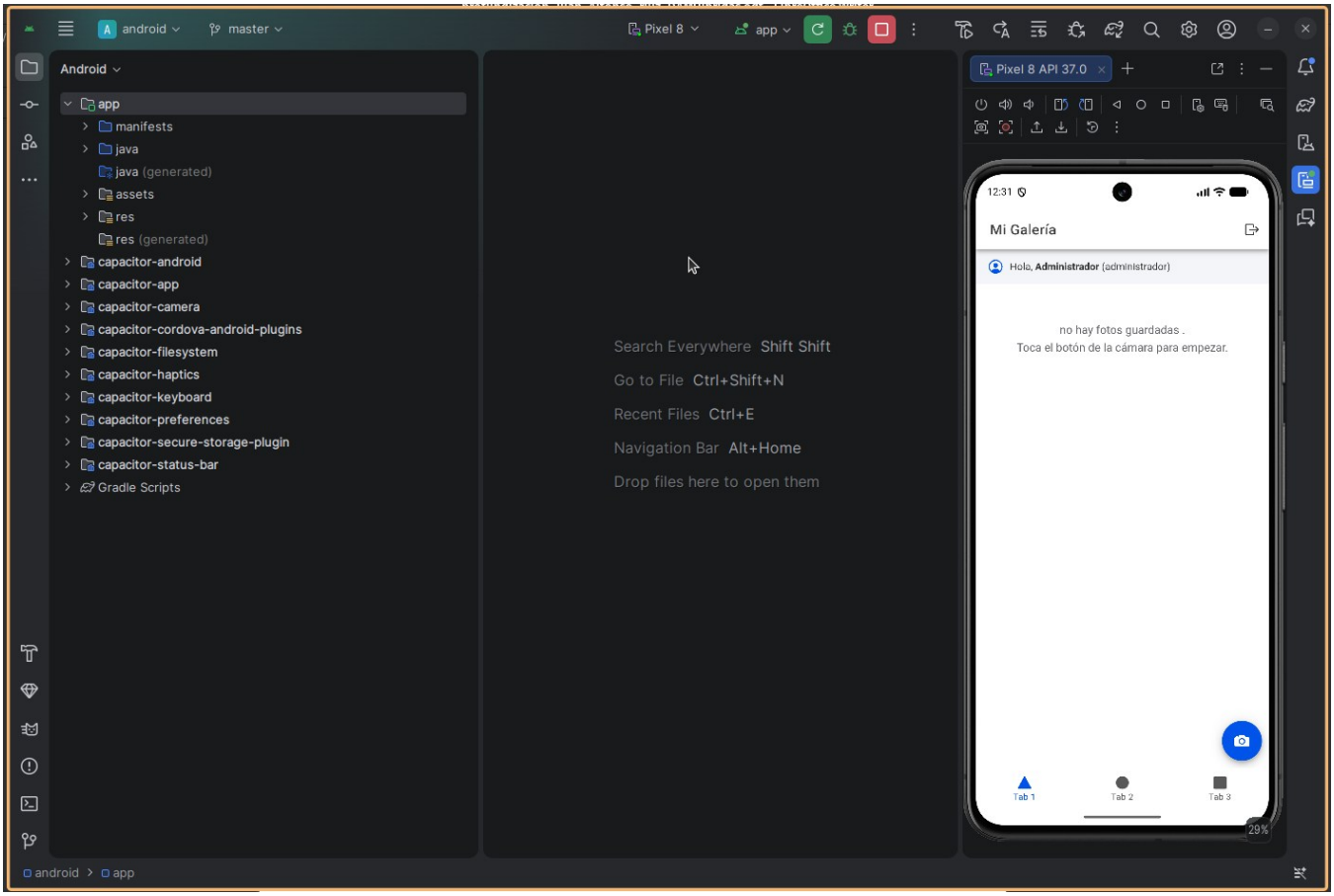
4.2.1 Ejecución de la aplicación

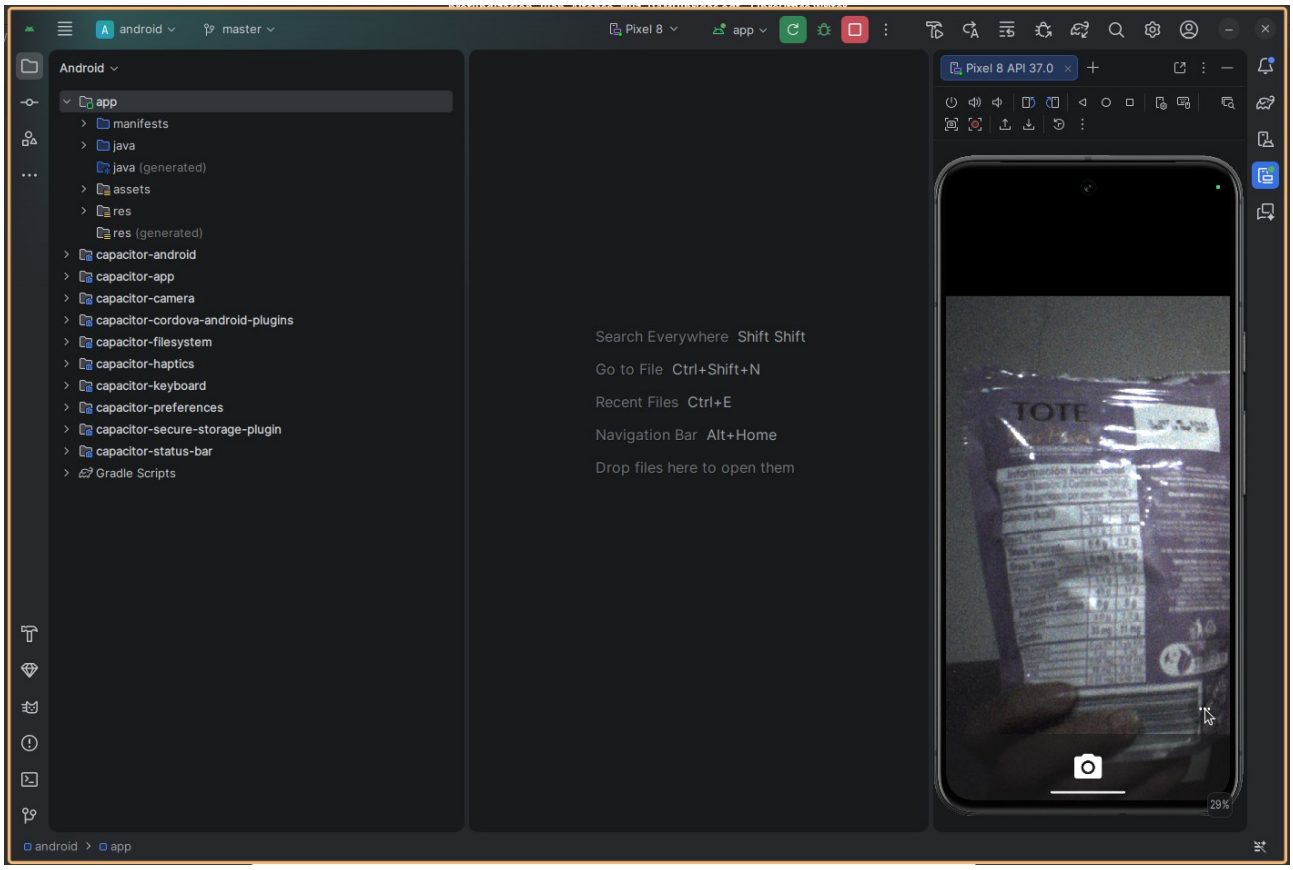
- Se presionó el botón "Run" () en Android Studio.
- Gradle compiló el APK e instaló la app en el emulador.
- La app se abrió automáticamente mostrando la pantalla de login.

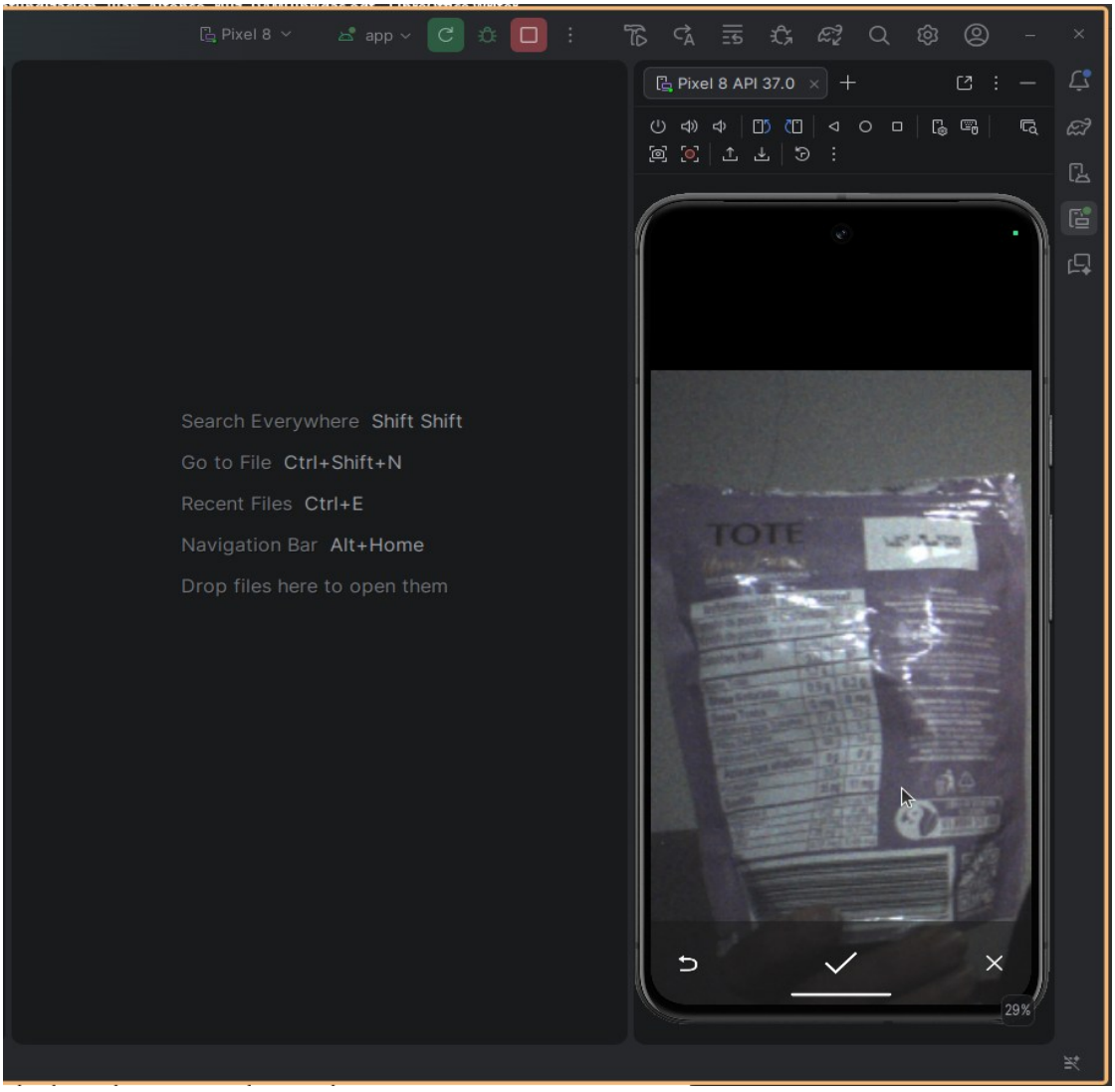


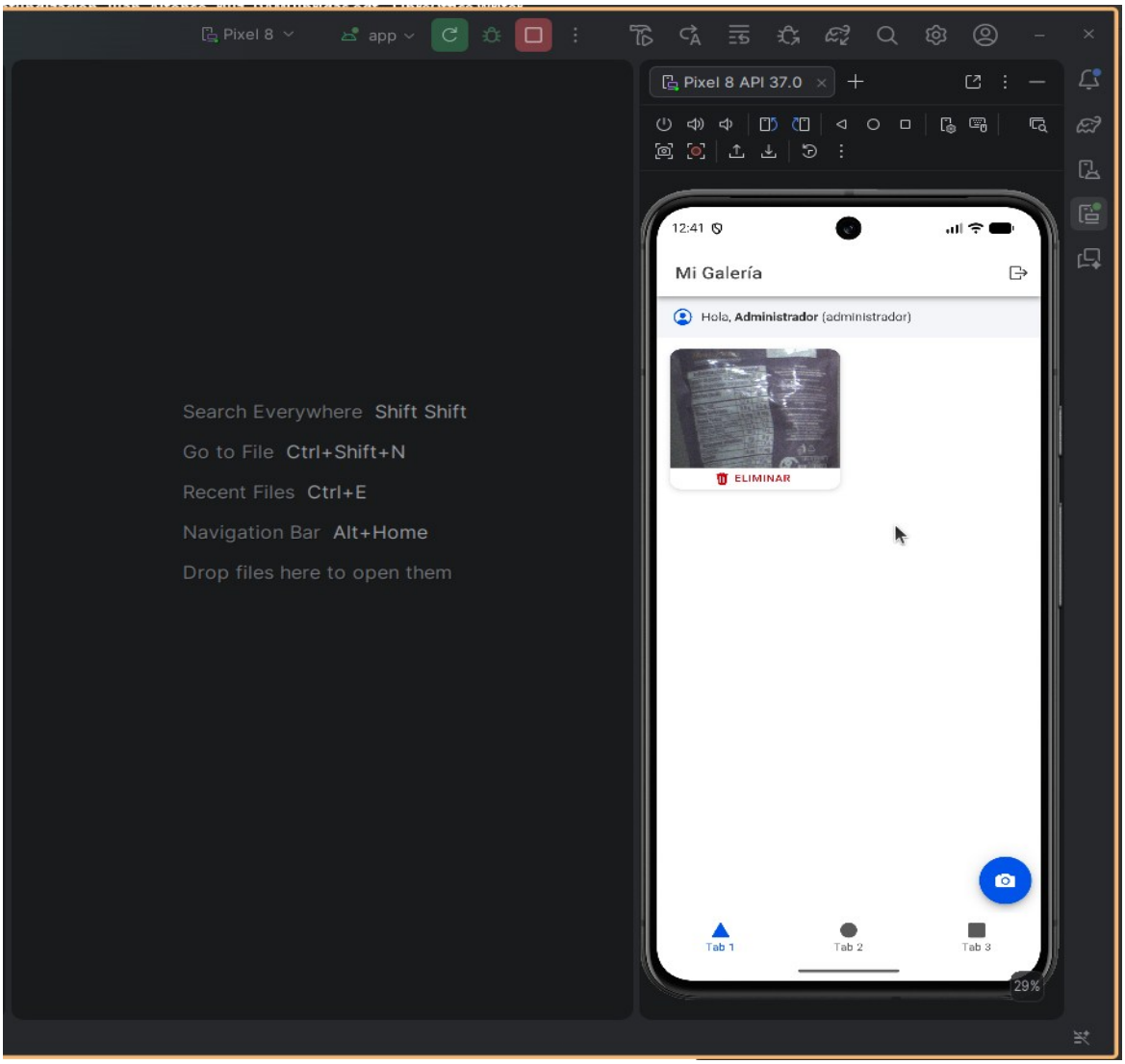
4.2.2 Prueba del flujo completo en Android

- Se hizo login con admin@test.com / 123456.
- Se accedió a la galería.
- Se hizo clic en el botón flotante de cámara.
- Se abrió la cámara nativa del emulador (con la escena virtual de Android o la webcam real si se configuró).
- Se capturó una foto y se aceptó.
- La foto apareció en la galería.
- Se cerró la app y se reabrió: las fotos persistieron.
- Se le probó el logout y se regresó al login.




android
master
Pixel 8
app
Android
app
manifests
java
java (generated)
assets
res
res (generated)
capacitor-android
capacitor-app
capacitor-camera
capacitor-cordova-android-plugins
capacitor-filesystem
capacitor-haptics
capacitor-keyboard
capacitor-preferences
capacitor-secure-storage-plugin
capacitor-status-bar
Gradle Scripts
Search Everywhere Shift Shift
Go to File Ctrl+Shift+N
Recent Files Ctrl+E
Navigation Bar Alt+Home
Drop files here to open them
Pixel 8 API 37.0
29%
android
app

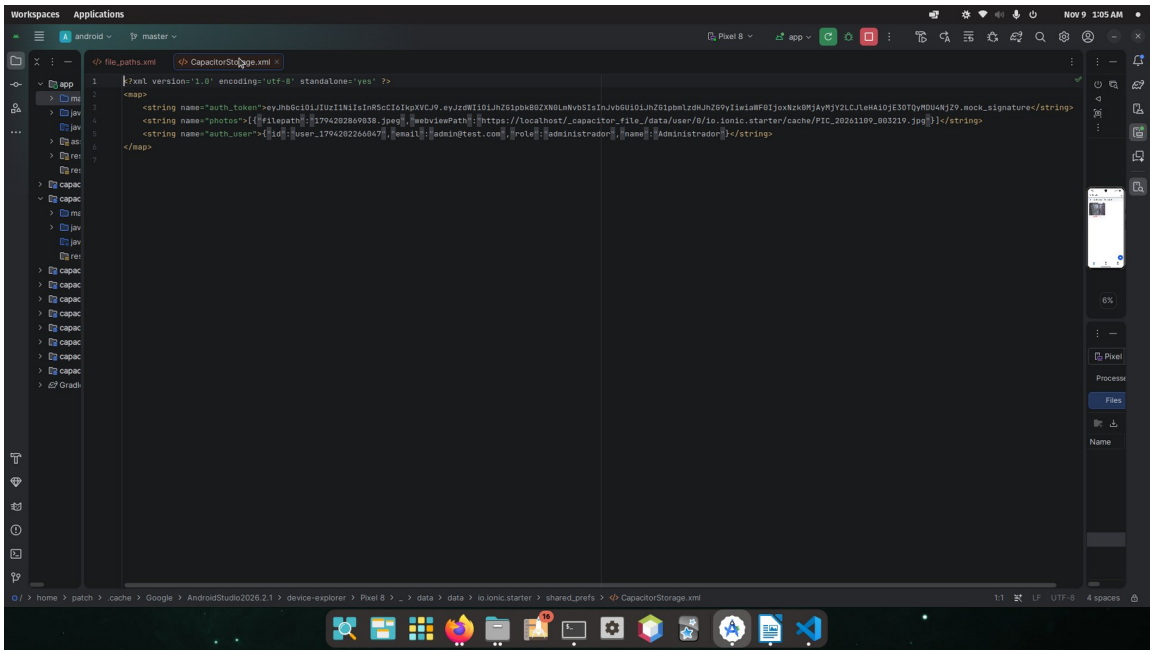

Pixel 8
app
Search Everywhere Shift Shift
Go to File Ctrl+Shift+N
Recent Files Ctrl+E
Navigation Bar Alt+Home
Drop files here to open them
Pixel 8 API 37.0
29%



4.2.3 Verificación del almacenamiento en Android

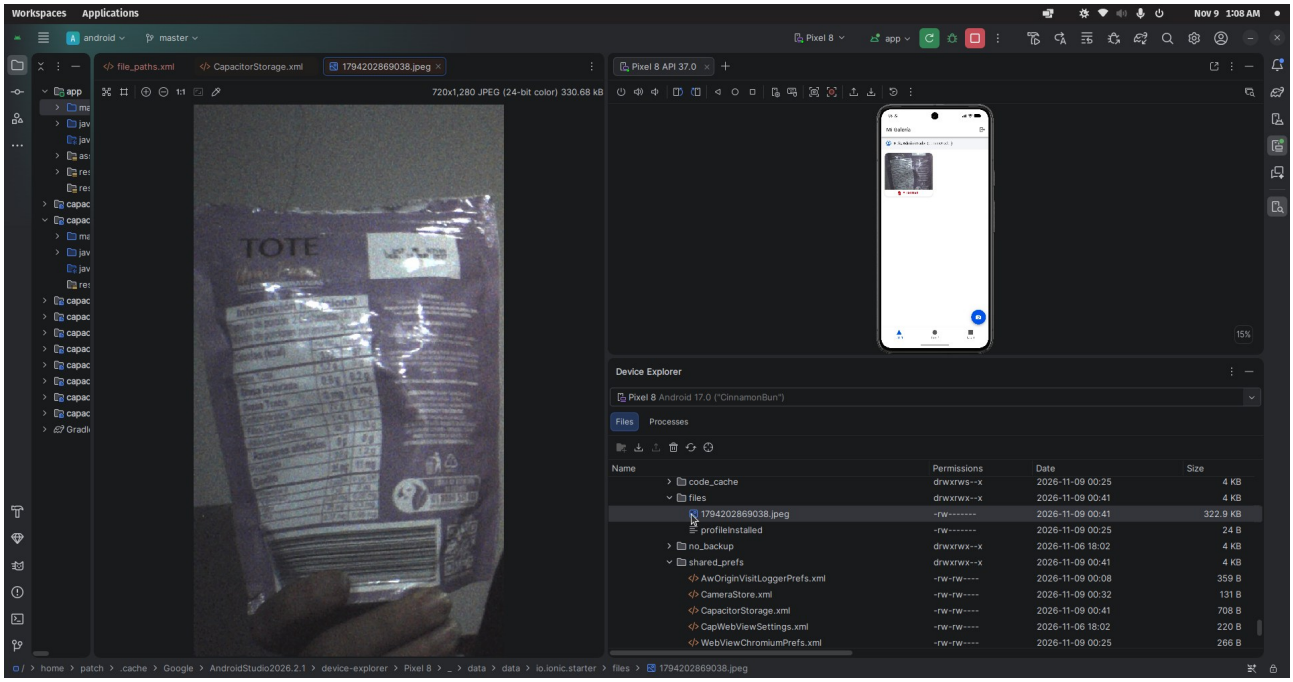
Se utilizó la herramienta Device Explorer de Android Studio para inspeccionar el almacenamiento interno de la app:

Preferences: en data/data/io.ionic.starter/shared_prefs/CapacitorStorage.xml se encontraron las claves auth_token, auth_user, registered_users y photos.





Fotos: en data/data/io.ionic.starter/files/ se encontraron los archivos .jpeg capturados.



5. CONCLUSIONES

El desarrollo de esta actividad permitió integrar de forma práctica los conceptos fundamentales del desarrollo de aplicaciones móviles híbridas con Ionic, Angular y Capacitor, aplicando una arquitectura profesional basada en servicios, inyección de dependencias y separación de responsabilidades.

Logros alcanzados:

Configuración completa del entorno: se instalaron y configuraron Node.js, Ionic CLI, Angular y Android Studio, resolviendo problemas de compatibilidad entre Flatpak, PATH, JDK y Gradle.

Implementación de la galería de fotos: se construyó un servicio (PhotoService) que encapsula la captura de imágenes con la cámara del dispositivo, el almacenamiento en el sistema de archivos y la persistencia de metadatos en Preferences. La galería funciona correctamente tanto en el navegador (con PWA Elements para la cámara web) como en Android (con la cámara nativa).

Arquitectura JWT funcional: se implementó un flujo completo de autenticación con:

- Servicio de autenticación (AuthService) con generación de JWT.
- nterceptor funcional que adjunta el token a cada petición HTTP.
- Guard de rutas que protege /tabs y redirige a /login.
- UI de login/registro con múltiples roles (comprador, vendedor, administrador).
- Logout que limpia la sesión y vuelve al login.

- Migración exitosa a Android: la aplicación se compiló y ejecutó correctamente en un emulador Android, validando tanto la cámara nativa como el almacenamiento persistente en SharedPreferences y en el sistema de archivos privado de la app.

Limitaciones identificadas:

Datos no sincronizados entre dispositivos: al usar almacenamiento local (Preferences + Filesystem), los usuarios y fotos solo existen en el dispositivo donde se crearon. Esta es una limitación inherente al mock y se resolverá en actividades posteriores con un backend central.

Contraseñas en texto plano: el AuthService guarda las contraseñas sin cifrar en Preferences. En un entorno de producción, esto sería inaceptable y debería reemplazarse por hashing con bcrypt o argon2 en el backend.

JWT con firma ficticia: el token generado tiene estructura válida pero no está firmado con una clave secreta real. Un backend real firmaría con HMAC-SHA256 y el cliente solo almacenaría el token firmado.

Trabajo futuro:

- Implementar el backend con Node.js + Express o Firebase para la sincronización de usuarios y fotos entre dispositivos.
- Integrar un sistema de notificaciones push para el estado de los pedidos (RNF del proyecto de tienda virtual).
- Añadir autenticación biométrica (reconocimiento facial o huella dactilar) como método alternativo de login.
- Migrar las contraseñas a un modelo de hashing seguro y el JWT a capacitor-secure-storage-plugin.

REFERENCIAS

Angular Framework. (s.f.). *Angular: The modern web developer's platform*. https://angular.dev

Biessek, A. (2019). *Flutter for Beginners: An introductory guide to building cross-platform mobile applications with Flutter and Dart 2*. Packt Publishing. https://www.packtpub.com/en-us/product/flutter-for-beginners-9781788996082

Hayward, J. S., Fedosejev, A., Prusty, N., Horton, A., Vice, R., Holmes, E., & Bray, T. (2016). *React: Building Modern Web Applications*. Packt Publishing. https://www.packtpub.com/en-us/product/react-building-modern-web-applications-9781786462268

Ionic Framework. (s.f.). *Introduction to Ionic*. Recuperado el 12 de julio de 2023 de https://ionicframework.com/docs

Khanna, R. (2016). *Getting Started with Ionic*. Packt Publishing. https://catalogue.packt.com/product/9781784390570

Lasa Gómez, C., Álvarez García, A., & Heras del Dedo, R. D. L. (2017). *Métodos ágiles: Scrum, Kanban, Lean*. Anaya Multimedia. https://www.google.com/books/edition/M%C3%A9todos_%C3%A1giles_Scrum_Kanban_Lean/3IY2DwAAQBAJ

Mejía Trejo, J. (2023). *Apps móviles sin programación: principales herramientas para crearlas*. Academia Mexicana de Investigación y Docencia en Innovación. https://elibro.net/ereader/siduncu/228101

Recio García, J. A. (2018). *HTML5, CSS3 y jQuery. Curso práctico*. Ediciones de la U. https://www.buscalibre.com.ar/libro-html5-css3-y-jquery-curso-practico/9789587627832/p/49161535